

Version Control for Code You Did Not Write

One edit per function, filed under the request that asked for it

Anonymous Author(s)

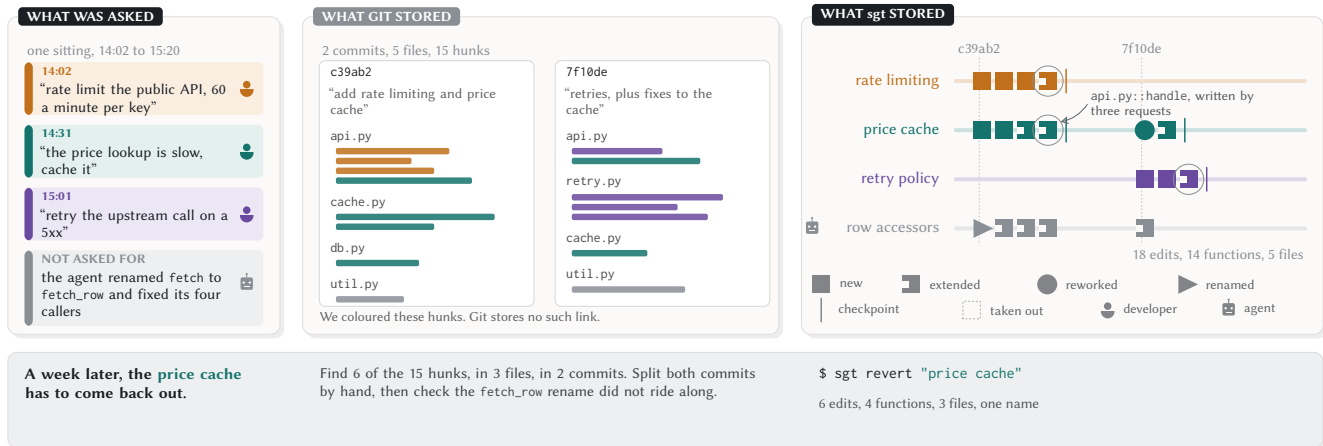


Figure 1: One hour with a coding agent, recorded two ways. The developer asked for three things and the agent did a fourth nobody asked for, and the two records differ in whether anything in the repository says which is which.

Abstract

A developer working through a coding agent specifies intent in sentences but version control records lines in commits. The developer who wants to undo one request must recover which functions it produced, a decomposition they stated before any code existed and nothing wrote down.

We present sgt, which records one edit per changed function, groups edits into named features by how they change together, and rebuilds files from whichever set is included. A version is a set of edits; removing a feature is set difference; restoring it is set union. The record lives inside an ordinary git repository and requires no server.

A pilot study found the split the evaluation predicts: a new user adopted the comprehension layer on first contact (attribution, dependency display, revert preview) and declined the write layer after one session, because three of four mutation commands failed. The comprehension layer told the developer what to change; the write layer could not yet be trusted to change it. We report both halves, state which measures would count against the design, and pre-register the study that would produce them.

Keywords

version control, coding agents, program comprehension, developer tools, change history

1 Introduction

A version control system records changes so they can be revisited and reversed. Every widely adopted system records lines of code—whether as snapshots [85] or as patches to those lines [59, 74]—and groups them into commits a developer names. This agreement dates to the earliest systems and has survived every redesign since, for a

reason that was sound from the start: the developer authored the code, held at least a rough model of what changed and why [53, 64], and could therefore supply the decomposition whenever a tool asked for it. Developers now specify intent in natural language and models translate it into edits across files [23]. Three controlled field experiments show this shift produces 26% more code per week with no measurable quality decline [19]. The condition that made the agreement sound—that the developer holds a model of what changed and why—no longer holds reliably. A developer who types three sentences in an hour and receives fourteen functions across five files cannot supply the decomposition a tool asks for, because they never held it. We argue that the design choice should be renegotiated.

Three consequences of this shift make the mismatch concrete. First, the unit that version control addresses, the line, is no longer stable enough to serve as an anchor. A model that generates a function in one response and regenerates it with different formatting or variable names in the next has changed every line while changing nothing a developer would name. Across 211 million changed lines from public repositories authored between 2020 and 2024, the share associated with refactoring—lines moved into a better position rather than written afresh—fell from a quarter to under a tenth, while code revised within two weeks of being written roughly doubled [30]. A line is now replaced faster than it settles into a position worth recording, and the developer wants to revisit a feature as it stood two requests ago—a unit the line-level record cannot express.

Second, the developer does not hold a detailed model of what the code says [76]. A natural language prompt specifies an outcome but not the logic that implements it, so the implementation contains decisions the model made and the developer has not reviewed. Barke et al. observed programmers reading generated code for

shape rather than line by line [5], Tang et al. found that developers often do not notice which code a model wrote and delete it rather than repair it [80], and among 410 developers surveyed by Liang et al. understanding generated code was one of the most frequently reported difficulties [56]. The consequence for version control is that the developer who searches a history for a prior state must describe it in terms they never acquired. This is the vocabulary problem Furnas et al. measured in information retrieval [28], now appearing inside the repository. Nguyen and Glassman observed it directly: beginning programmers and code models systematically misread each other's vocabulary, so a developer's search terms and the model's naming choices diverge even when both refer to the same behaviour [66]. The developer searches with vocabulary they never learned and finds whatever happens to match instead of the version that served a stated purpose [18].

Third, the unit the developer thinks in has moved from a location in the code to a purpose the code serves. A prompt names what the software should do (rather than which file to edit), so the developer's mental model of their own work is organised by feature: the retry policy, the export endpoint, the rate limiter. This is the level the program comprehension literature has always found people work at: Biggerstaff et al. called recovering which functions implement a named concern the concept assignment problem [8], and the feature location field exists because answering it by reading code is expensive [22]. The commit is the closest unit a version control system offers, and it does not correspond to this level, since Herzig and Zeller found changes serving several unrelated purposes throughout the histories of five open source Java projects [42]. An agent session produces exactly that structure, because a commit ends where the developer stopped to type a command and a request ends somewhere else.

The response we describe is a version control system that records at the grain the developer thinks at. Instead of recording lines inside a commit, we parse each file into its functions and classes [10] and record one edit per changed function, together with what that edit was built on. We group those functions into features from how they change and how they refer to one another [87, 96, 101], and label each group with the words the developer and the agent used while the work was happening. A version of the code is then a set of these edits, and the files on disk are rebuilt from whichever set is included, so a feature can be taken out or put back by naming it rather than by naming the lines it produced. None of the individual techniques is new. Change coupling mining, community detection, feature location, and set-based version control each have a literature. What is new is composing them into a single tool that a developer operates daily, because that composition introduces interactions none of the original designs anticipated. A function identity tracker that loses a rename breaks the dependency chain a revert depends on. A grouping inference that absorbs a shared helper pulls unrelated work into a single feature. A layout record (the whitespace between functions) that becomes individually addressable acquires failure modes the entity record does not have, and 68% of the violations we found in ten thousand operations entered through that seam. The contribution is in making the composition survive daily use on a real codebase, in measuring where it fails honestly, and in reporting both the design's costs and the conditions under which it should not be adopted.

For example, a service that answers price quotes for a trading dashboard has a handler, a database layer, and a set of row accessors. On Monday the developer prompts cache the price lookup, it takes 400ms and the price barely moves, and the agent writes `cache.py::get_cached` and `cache.py::_ttl`, gives `db.py::query` a parameter for how long a cached price stays good so the cache can pass one through, and changes `api.py::handle` to consult the cache before the database. That is four edits to four functions in three files, and the two in `cache.py` share no line with the one in `api.py`, yet all four are filed under one name. Over the next two days two further prompts change `api.py::handle` again, one adding a rate limit and one adding a retry policy, and while it is adding the retry the agent goes back and reworks two of the cache's own functions, which brings the price cache to six edits. A colleague adds a metric for quote latency. Three weeks later the developer finds the dashboard showing a price sixty seconds stale, so they type `sgt revert "price cache"`. `sgt` prints the six edits that would come out, names `api.py::handle` as the one function where later work was built on something being removed, shows the text each change would produce, and asks before touching anything. The rate limit and the retry policy stay, because their edits are still included; the colleague's metric stays, because it was never in the set coming out; and the parameter leaves both `db.py::query` and `api.py::handle`, because both were in it. When the developer decides a week later that the cache was worth keeping after all, `sgt restore` returns the files byte for byte, because putting the six edits back is the same set arithmetic as taking them out and nothing is replayed as a patch. Figure 1 draws these three requests arriving in a single sitting, under both records, and Section 5 follows this repository through five sittings.

Three prices are paid for this, all of them in the first hour of use rather than in some later edge case. Two edits to the same function made from the same starting point are recorded as competing versions that a person has to decide between, including the pair at opposite ends of a fifty line function that touch no common line and that git would have merged without comment. The grouping of edits into named requests is inferred, so it is a guess and it will sometimes be wrong, and correcting it costs the developer a rename, a merge, or a split, none of which moves a recorded edit. `sgt` reads Python, TypeScript, and TSX and versions every other file whole, at the grain git already uses, so a repository whose interesting logic is written in Go gets nothing from `sgt` today.

The paper makes four contributions. We characterise the mismatch between the unit a developer using an agent decides in and the unit a version control system records, one attempted operation at a time (Section 3), in the style of conceptual analysis Pérez De Rosso and Jackson applied to git itself [69, 70]. We describe `sgt`'s design as four commitments, say what each one buys and what it costs, and run the whole set of operations on one repository (Sections 4 and 5). We state which observation would show each of our claims to be wrong, and pre-register the study that would produce it (Section 6). We report what fifty-seven days of `sgt` versioning its own construction taught us about where the design breaks (Section 9), and what three pilots taught us about the distance between a property that holds across 10,237 operations and a property a new user can rely on (Section 7).

2 Related Work

2.1 Programming by request

The ways to program with foundation models have changed constantly over the last four years, along with the advance in what the models can do. At first a model was mostly a programming assistant offering inline code completion. Barke et al. watched 20 programmers and found them using it in two ways, finishing a line they had already planned, or asking for a starting point in code they did not yet know how to write [5]. Vaithilingam et al. found that 24 participants using Copilot did not complete tasks faster and still preferred to keep it on, because it gave them a draft to work from [89], and Sarkar et al. collected what the experience felt like from the developer's side [76]. In all of these the developer accepted a line or a block of code. Four years later SWE-bench gives a model a natural language issue report from a real repository and scores it on whether the repository's own tests pass afterwards [47], and the systems built to score well on it edit several files in a loop [95]. The unit a developer hands over has gone from a line to a paragraph of English, and the code that comes back is correspondingly larger.

All of this work studies the minutes around acceptance: whether the code is correct [71], whether the developer can tell that it is correct [90], whether they trust it [14, 56], and how much faster they finish [19, 63]. Nguyen and Glassman studied what happens during acceptance itself and found a systematic vocabulary mismatch: beginning programmers and code models read each other's intent through different naming conventions, so the developer's description of what they asked for and the model's description of what it produced diverge even when the behaviour matches [66]. That mismatch compounds over time, because the developer who later searches their history must use the model's naming, not their own, and cannot tell when the two have drifted apart. None of this work asks what happens a week later, when the developer needs to recall what their codebase was, what has changed since, and why it is constructed the way it is. Bainbridge identified this structure in 1983: automation removes the need for a skill, the skill degrades through disuse, and when the automation fails the operator needs exactly the skill that has degraded [4]. The irony applies here directly. A developer who specifies in natural language no longer practises the decomposition that version control demands, so the ability to say which function served which request—the skill git's revert requires—decays in proportion to the success of the tool that made the decomposition unnecessary. Bastani et al. confirmed the mechanism experimentally: participants who used a generative model during practice solved faster in the moment but performed significantly worse on later assessments without access to it [7]. We observe programmers gradually losing the theory of their own codebase [64] over the time they use these tools, and Bainbridge's framework says the loss is structural rather than incidental: it is what automation does to the skill it replaces.

2.2 What a version control system agrees to record

Git records a version as a snapshot of the whole working tree, and each commit names its parents plus a tree of file contents addressed by hash [85]. Verifying that a history has not been altered is then a

matter of rehashing it, and a merge is computed on demand from a common ancestor, so nothing in the repository has to say what happened when two edits met. We keep the first of those properties and give up the second on purpose. sgt addresses its records by their contents, and when two edits to one function descend from the same parent it records that they did rather than computing what the pair amounts to, for the reason Section 4.3 gives.

Pérez De Rosso and Jackson analysed git's concepts against the purposes users bring to them, found several places where a concept serves a purpose users do not have, and removed the staging area and detached HEAD in their redesign [69, 70]. The commit survives that redesign untouched, and it should, because it serves a purpose users do have. It is also the smallest thing git can address, so any operation over less than a commit works by having the developer point at lines, and git add -p and git rebase -i are the tools for pointing. Under git's original constraints this is the cheapest correct design available, because the person running git add -p in 2005 had typed the lines minutes earlier, and writing their decomposition into the repository would have meant maintaining a second structure that could disagree with the first.

Darcs takes the other option and records a version as a patch that adds and removes lines in a stated context [74]. Patches that do not depend on each other commute, so a user can pull the third patch in a series without the first two, and the design has been formalised and reimplemented since on a data structure that makes the operations fast [46, 59, 61]. A Darcs user who pulls one patch onto another branch moves the patch that was recorded, along with the dependencies it names, where git's cherry-pick reapplies a diff in a context it was not written against and produces a second commit with a second hash, so the same work has two identities. But Darcs works out what a patch depends on from the lines the patch touches, so a dependency that leaves no trace in the text goes unrecorded. When an agent adds a Retry class in a new file and edits api.py::handle to call it, the two patches share no line, and nothing in the record says that pulling the second without the first yields a file referring to a name that does not exist. Darcs is explicit about this and offers a way to declare such a dependency by hand, and we would have made the same call in 2003, because computing a dependency between two functions requires parsing the language and a system that parses works for some languages and not for others. Darcs declined that price and we pay it.

Jujutsu and Mercurial both add a record of history's own history. Jujutsu gives each change an identifier separate from its commit hash, so rewriting a commit produces a new commit while the change keeps its name, and a change with more than one visible commit is called divergent [91, 92]. Mercurial stores obsolescence markers apart from changeset data, keeping a history of how the history was edited alongside the history of how the files were [83]. Both designs let a developer say "the change I was working on yesterday" after five rebases and be understood, and both let two people rewrite the same region of history and have the disagreement recorded rather than silently resolved, which is the construction sgt uses for two competing versions of one function. We part company on what receives the durable identity. In both systems it is a commit, so the identity is exactly as coarse as the commit, and after a developer squashes a two-hour agent session into one commit,

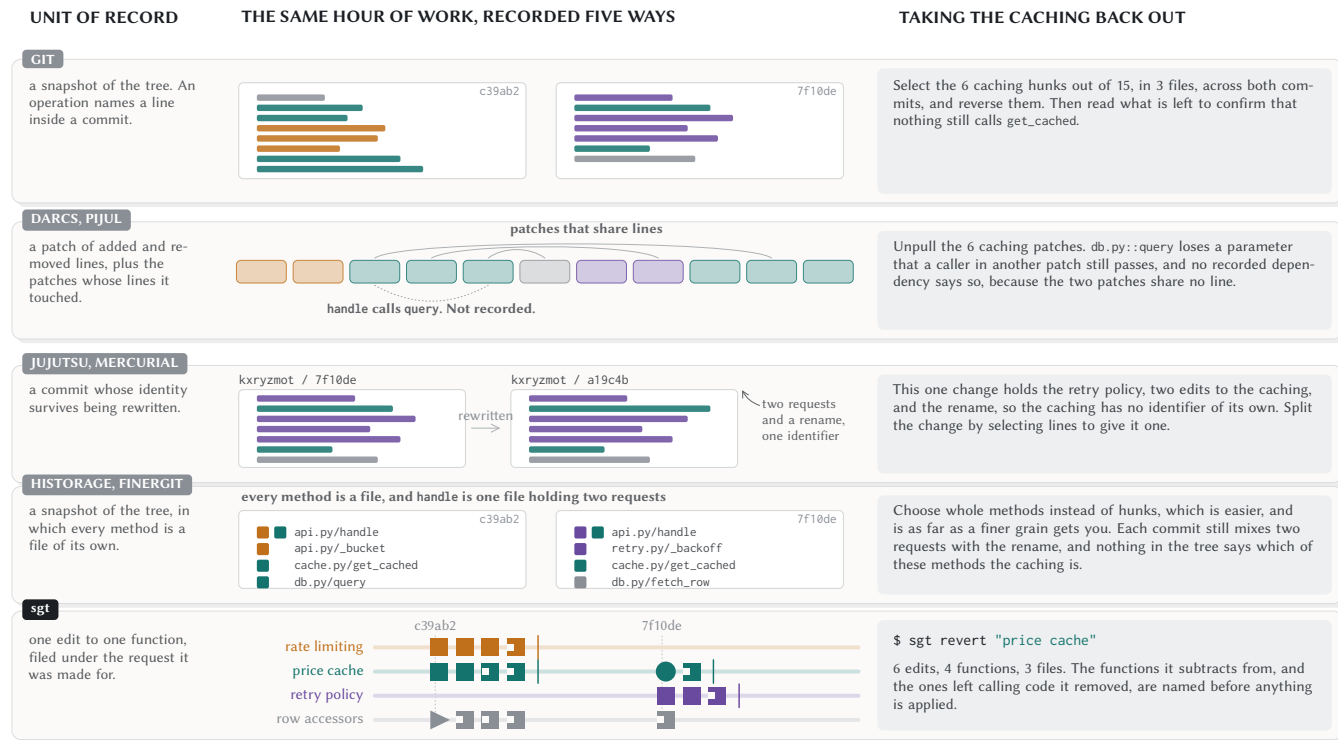


Figure 2: One hour of work, recorded five ways, with the same request taken back out of each. The fourth row already records at the grain of a method and still cannot say which of those methods the caching is, which is why we read what is missing as the name of the request and not the size of the unit.

one change identifier covers all three requests and nothing smaller has a name. Figure 2 puts these records beside each other.

2.3 Version control that already knew about functions

Recording a version at the grain of a function has been built before, more than once, and the reason none of it displaced the commit is the reason we think it can now. Historage rewrites a Java repository so that every method sits in a file of its own, at which point git's existing machinery gives each method a history of its own [38], and FinerGit refines that format so that git's rename detection keeps working when a method moves [43]. Stellation and Coven made the fine-grained unit the thing the repository stores rather than a transformation applied to it, so a developer could check out a single method [15, 16]. MolhadoRef settled the problem Section 9.2 reports as our weakest link, by having the development environment record each refactoring as the developer performed it, so a rename entered the history as an operation instead of arriving as something a later tool had to guess at [21]. Mylyn recorded, for each task a developer worked on, the set of program elements they touched while working on it, and used that set to decide what the environment showed them [49]. All of this existed at least fifteen years before the first coding agent and all of it worked. Historage and FinerGit are in use today as instruments for mining software histories, which is research reading a repository from the outside, and a working

programmer choosing what to run on Monday morning uses none of it.

We do not read that as a failure of engineering, because a finer grain buys a developer only what they cannot already supply for themselves, and until recently they could supply all of it. Whoever wanted the caching out of a 2010 repository had written the caching, so a system that told them which methods it touched was performing work they had already done, at the price of a repository their colleagues could not clone with plain git. The value of a finer grain has not changed; what changed is the identity of the author. A record that was redundant while the developer wrote the lines is now the only surviving copy of the decomposition, because the developer has stopped writing them. Mylyn's task context and sgt's named group of edits are the same object built for two different situations, and the difference is that Mylyn watched a developer's attention move through code that developer was writing, whereas the attention sgt reconstructs belonged to a process that has already exited.

The merge question in this literature was approached from the end opposite to ours. Horwitz, Prins and Reps showed that two versions of a program can be integrated automatically when a semantic analysis proves the changes do not interfere [44], and Apel et al.'s semistructured merge parses a file far enough to see that two edits landed in different methods, resolving a large share of what a line-based merge reports as a conflict [1]. The design here rests directly on two further results from that area, that a

merge of the recorded operations tells a developer more than a merge of the states those operations produced [57], and that a version is usefully treated as a set with a logic over it, which is the construction Section 4.2 uses [100]. sgt departs from Horwitz and from Apel deliberately. Where semistructured merge combines two edits to one method as soon as it can see they touch different lines, sgt stops and asks a person, and Section 9.5 concedes that a repository with one person in it is the wrong place to have tested that rule.

2.4 Recovering a decomposition after the fact

Commits that address more than one concern distort the tools built from a history. Herzig and Zeller examined five open source Java projects, found such commits throughout them, and showed that separating those changes alters which files a defect prediction model identifies as the most likely to contain bugs [41, 42]. The untangling tools built after that finding recover the grouping from the diff itself, partitioning it by the definition and use relationships between the changed statements [6] or by a dependency graph across the versions a commit spans [72], and every one of them is evaluated on whether it reproduces a grouping a human labelled. Two anticipate the move we make: Dias et al. read the fine-grained edit events an IDE records while the developer is still typing [20], and Shen et al. hand the partition back to the developer to correct, which is the arrangement Section 4.4 adopts [78, 81, 93]. What none of them could assume is a statement of the boundary written down before the code existed, because in 2015 no such statement existed to be had, and the strength of their inference is a measure of how much they had to recover without one.

That is the loosened constraint, and it is worth being exact about what it buys, because sgt performs the same inference they do and performs it no better; we would expect it to be worse, since it has to finish while a developer waits. What has changed is what the inference is for. Their partition has to be right, because it is the answer. Ours has to be close enough that a name attaches to the right set, and the developer who reads a wrong name fixes it with a rename that moves no code.

sgt does depend on two results from this literature. Keeping a function's identity when it is renamed or moved is the problem that refactoring detection and tree differencing address [25, 27, 88], and it is the weakest link in our implementation, since a rename we fail to detect appears as one function removed and another added. Grouping edits into features is a clustering over which functions change together and which call each other, both halves of which the co-change mining literature established [32, 52, 96, 101], resolved with a community detection algorithm [86, 87]. Both are inferences, and Section 4 states what a developer does when they are wrong.

2.5 Histories the editor keeps

A second tradition puts the record in the editor and asks nothing of the developer. Azurite stores a programmer's editing operations at a finer grain than any commit and lets them undo one past edit while keeping the edits made after it [97–99]. Variolite lets a data scientist wrap a region of code as a variant and switch between variants in place, after Kery et al. found that people doing exploratory work avoid version control for exactly that work [50],

and the same tradition has held alternative versions side by side, replayed a document's editing history, versioned one experiment at a time, and cleaned a notebook up afterwards, in every case recording what the developer did as a byproduct of their doing it and never asking them to prepare it [35, 37, 39, 51, 60, 82]. That commitment sgt keeps. All of them also stop at the boundary of one machine: the history lives in the editor's own store, so a colleague, a build, and a code review never see it, and the unit is a single edit event, and a set of related edits still has no name a person can use. sgt crosses that boundary by writing its record into the git repository as files committed with the code, and Section 9.4 describes what that costs.

2.6 Layers over git built for the way people work now

The tools developers reach for today accept git's storage and add a layer above it. GitButler keeps several branches applied to one working directory at once, draws each as a vertical lane with its own staging area, and lets the developer drag a change into the lane it belongs to [29], on the correct reasoning that which branch a piece of work belongs to is easiest to decide once the work exists. GitButler assigns an individual file change to a lane, so it cannot express the case at the centre of Figure 1, where three requests all edited `api.py::handle` and no assignment of `api.py` to one lane is right. Graphite and GitHub's stacked pull requests take the ordered series as their unit, keeping a chain of changes consistent by rebasing and retargeting the higher layers when a lower one merges and landing every unmerged layer beneath the topmost ready one in a single operation [31, 33], which is the right answer to a reviewer who cannot read one large change with the care they can give to each of five small ones. Graphite's guide states the precondition plainly, that developers must understand how to break down their work into small, incremental changes, and every command the tool offers operates on a decomposition the developer has already performed. In an agent session that decomposition exists in the developer's request history and nowhere in the code.

The closest system to ours is sem, which parses about 32 languages with tree-sitter and reports which functions, methods and classes a commit changed, resolving an entity's identity across versions in three passes, by exact identifier, by a structural hash that ignores whitespace and comments, and by token overlap above 80% [2]. sem keeps its answers in a derived SQLite cache outside the repository, and that placement makes sem safe to adopt and safe to abandon, gives entity-level answers about a history recorded years before sem existed, and requires agreement from nobody else on the team. It also bounds what sem can offer: a record any team-mate's ordinary commit can silently invalidate cannot be the record an operation acts on, so every sem command reports and none of them changes state, and a developer who learns from sem diff that `get_cached` changed removes it with a line-level tool. sgt commits its record to the repository in order to act on it, and we pay for that in two places. A record that operations depend on has to be repaired when it is wrong, which Section 9.3 reports the current cost of, and everybody who pulls the repository now carries a record they did not ask for, which Section 9.4 works through from the point of view of the three people who never installed sgt. sem chose the safer

placement and we chose the more useful one, and the honest form of that sentence is that we have taken on an obligation sem does not have.

3 Five Operations a Developer Cannot Name

All five scenarios below fail for the same reason: the developer names their work by the request that produced it (“the caching,” “the rate limiter”), because that is the level their understanding of the code is pitched at [64], while the record they have to operate through holds lines inside commits (Figure 2). Every scenario has a workaround and all the workarounds work. The argument is about what the workarounds cost, and what they cost is recovering from the code which function served which request—a decomposition the developer stated in their own words before any code existed, and that nothing wrote down.

The five scenarios follow one developer through a single afternoon and the week after it. The project is a Python service that answers price quotes for a trading dashboard; its request handling lives in `api.py::handle`, its database access in `db.py::query`, and the developer asked a coding agent for three things in one sitting: a rate limiter, a price cache, and a retry policy.

3.1 “Commit this when it is done, and I do not know yet when that is”

A developer working through an agent still has to choose when to commit, and in this hour every moment available to them is wrong in a different way. Committing after each request writes a version of `api.py::handle` into the permanent history that existed for twenty-nine minutes and that nobody ever ran the test suite against, and it files the agent’s rework of the cache under the retry commit, because the agent did that rework while adding the retry. Committing once at the end produces one commit holding all three requests and the rename, and this developer split the difference by committing twice, which is the pair Figure 1 draws, each of them mixing two requests with the rename. Staging hunk by hunk produces four clean commits and asks the developer to decide, for each of fifteen hunks, which of those four purposes it serves. Green and Petre named the first two costs premature commitment, where a notation forces a decision before the information it needs exists, and the third viscosity, the resistance a notation puts up once a decision turns out to be wrong [9, 34]. Git asks for the decomposition at the moment of writing, and in a session driven by requests the boundary does exist at that moment. The developer stated the boundary themselves before any of the code existed. What they lack is the list of functions on either side of it, which only the agent ever had.

The operation. Record the boundary the developer already stated, without asking them to state it a second time in a different notation.

3.2 “Change the rate limiter I built yesterday”

The rate limiting from 14:02 used a fixed one-minute window, and three days later it needs a sliding one. The work lands in two of the four functions the original request produced, and the developer’s first question is what the rate limiting looked like before they touched it. Git can follow one function at a time: `git log -L :handle:api.py` follows a function by matching a name pattern and

`git log -S` finds the commits where a string appeared or disappeared, so the developer runs the first command four times, once per function they believe was involved, and assembles the four answers themselves. The set has no name, so nothing checks whether four was the right number. Codoban et al. asked developers why they examine history and found the most common reason to be understanding the rationale behind a piece of code [18], and LaToza and Myers found questions about why code is the way it is among the hardest developers report answering [54]. The rationale in this case was one sentence of English, typed three days ago into a window that is now closed.

The operation. Name the work one request produced, see its current state and its previous state, and change it as a unit.

3.3 “Take the caching back out and leave the rest alone”

The caching turned out to serve a stale price for up to sixty seconds, and it has to go. Its six edits are spread over three files and two commits, mixed with nine hunks belonging to three other purposes. One cache edit adds a parameter to `db.py::query`; another passes that parameter from `api.py::handle`. Three functions the rename rewrote afterwards call a helper the caching introduced. `git revert` takes back a whole commit, so reverting either of these two removes work the developer is keeping. The developer therefore checks out both commits, selects the six hunks by hand, reverses them, and then confirms that removing the parameter did not break `handle` and that the three functions the rename rewrote no longer call `get_cached`. The confirmation takes the time, because the record contains no statement that the `db.py::query` signature and the `handle` call site were changed for the same reason.

The operation. Remove a named set of edits, leave every other edit to the same files in place, and name the code that still depends on the removed set before anything is applied.

3.4 “Send just the export work to the release branch”

An export endpoint has to ship today, and it was written on a branch that also carries a half finished retry policy. `git cherry-pick` moves a commit, and the export work is two functions in a commit that also carries part of that retry policy, so the usual answer is a temporary branch and an interactive rebase that splits that commit, and the split has to be done again if more export work arrives before the release. Darcs solves the part of this that is about lines, since a Darcs patch carries the dependencies it was recorded with and moving it involves no reapplication (Section 2). But the export code calls a helper the rate limiter introduced, no line-level record states that call, and the developer finds out when the release branch fails to import.

The operation. Carry a named set of edits and its dependencies to another branch, and be told before the operation which dependencies would come along.

3.5 “Let someone review the rate limiter while I keep building on it”

The rate limiter was finished at 14:30 and the caching that followed calls `api.py::bucket`, so the two form a stack of two layers with the rate limiter underneath. Submitting them that way required the rate limiter to be on its own branch at 14:30, which required knowing at 14:30 that the caching would build on it. Stacked pull requests exist because review works best on small changes arriving often [73] and because a reviewer’s main difficulty is understanding the change in front of them [3], and Graphite and GitHub automate every step after the decision, including rebasing the upper layer when a reviewer asks for a change to the lower one (Section 2). The decision itself belongs to the developer and has to be made before the work exists, and in an agent session the layering becomes visible only once both layers are written. Green and Petre’s framework has a name for this, the hidden dependency, a relationship the system relies on that the notation does not show, and Church et al. traced its effects through version control in particular [17, 34]. The notation here is the branch, and a branch says nothing about which functions call which.

The operation. Discover the layering after the work exists, and submit one layer while continuing to work on the layer above it.

3.6 What the five have in common

Every one of these operations takes a set of edits as its argument, and the developer identifies that set by the purpose the edits served. Git expresses a set of lines, a set of files, and a set of commits, and a developer can build all five operations out of those three. The price of building them is recovering a decomposition that nobody recorded, and that recovery was affordable when the person paying it was the person who had written the code—because they held the answer already, having typed the lines minutes earlier. A developer who typed “add a retry policy” instead does not hold the answer, and the longer the session ran, the less they remember of the code it produced, until the recovery costs more than the operation it serves. Section 4 describes what we changed so that a developer can type the argument these five operations want.

4 What sgt Commits To

A version control system’s unit of record decides which questions a developer can put to their own history, and the questions a developer actually has are the ones their understanding of the codebase produces, so the unit should be the one that understanding is already organised around. sgt records one edit per changed function and files it under the request the edit was made for. This section states that commitment and the three that follow from it, each with the price it carries. Figure 3 shows the first two of them at work on the sitting from Figure 1: what one save writes down, and what a version is.

The design philosophy is that a tool should refuse rather than guess wrong, and should report rather than act silently. A differently-minded designer would merge concurrent edits to one function the way git merges concurrent edits to one file, on the grounds that most such merges produce working code. We tried that first and abandoned it because when the merge fails the developer has lost work they did not know was at risk, and the

failure appears somewhere other than where the merge happened. Another reasonable design would ask the developer at save time which request each function belongs to. We tried that too, and the developer cannot answer it without reading the diff, which is the labour the tool was supposed to eliminate. The four commitments below are what survived: each one is the design we arrived at after building the alternative and finding its cost.

4.1 One edit to one function, filed under the request

A save reads the working files, compares them against the last recorded state, and writes one record per changed function, naming the function, the version of it the edit started from, the version it produced, and those other functions the new code refers to that sgt can identify without guessing: a reference is recorded when the name it uses resolves to exactly one function in the codebase, and left unrecorded when the same short name is defined in several places, because a reference attributed to the wrong definition is worse than a missing one. The developer supplies one sentence, `sgt save -m "retry the upstream call when it returns a 5xx"`, and sgt reads everything else out of the files, so a record can be no more wrong than the code it was read from.

We chose the function as the unit because the three alternatives each fail on the same ordinary case. The price cache in Figure 3 added a parameter to `db.py::query` and passed a matching argument from `api.py::handle`, and those two changes share no line and no file, so a record that bottoms out at a line has nowhere to write down that they were made for one reason, and the developer who removes the cache a week later learns about the second one when the import fails. A record whose unit is the file cannot say that two changes to `api.py` served different purposes, which is the default case in Figure 1 where three requests all edit the same handler. GitButler uses this unit and solves the problem it can solve, which is the case where one feature per file suffices [29]. A record whose unit is the commit holds whatever was pending when the developer typed the command, which in Figure 1 is three requests and a rename the agent did on its own initiative, divided across two commits by the clock. A function is the smallest thing that both the developer and the agent name out loud while the work is happening, and the feature location literature works at that same unit because the code serving one concern is a set of functions scattered across files [8, 22].

sgt finds functions in Python, TypeScript, and TSX with tree-sitter [10] and records every other file as its whole content, at the grain git already offers, so the rest of this section applies only to files sgt can parse. Within the three parsed languages a function can be smaller than a purpose: the rename in Figure 1 moved `db.py::fetch` to `fetch_row` and updated four call sites, so sgt writes five records for what the developer thinks of as one action, and because those five are one named group the mismatch shows up only when somebody reads the edit list underneath the name. A function can also be larger than a purpose, which happens when two requests change opposite ends of one fifty line function, and Section 4.3 describes what sgt does then.

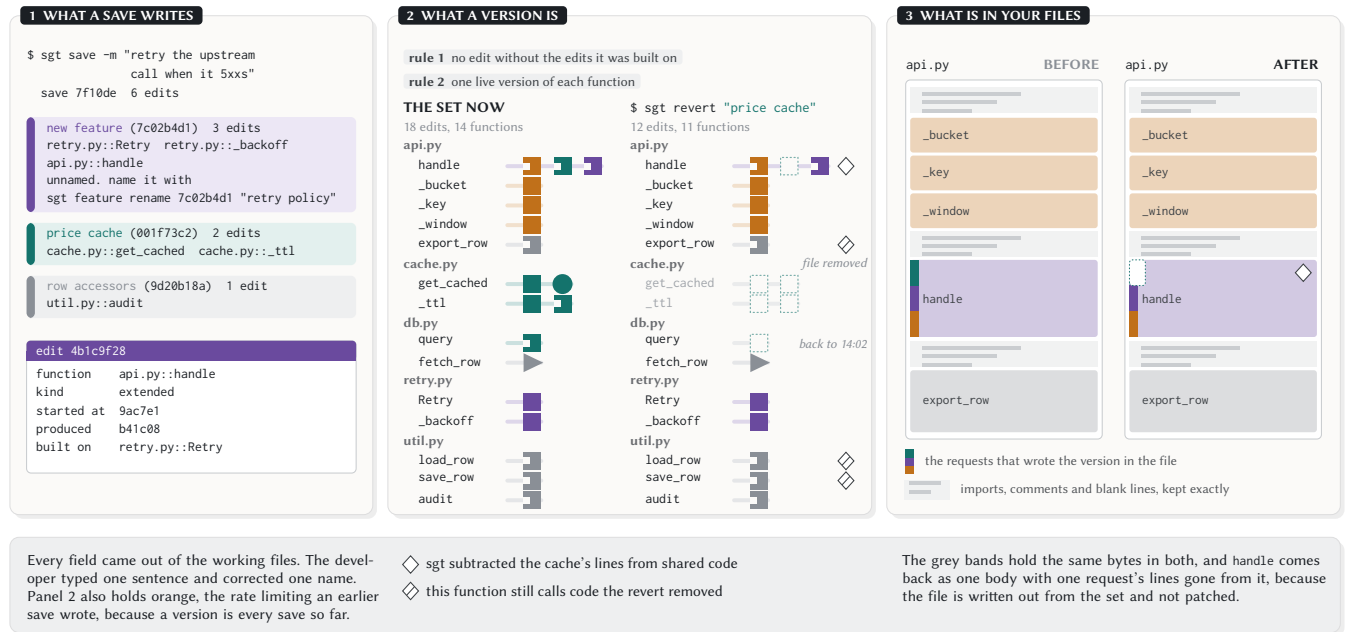


Figure 3: What one save writes down, what a version is, and where the bytes in a file come from. Every operation in this paper is arithmetic on the set in the middle panel, and the right panel is what that arithmetic does to a file the developer has open.

4.2 A version is a set of edits, and the files are rebuilt from it

Two rules decide which sets are versions. No edit is included without the edits it was built on, and each function has one included version. sgt rebuilds each file by writing out the included version of every function it contains and keeping the text between functions exactly as it was, which produces the bytes that are checked out. The middle panel of Figure 3 is one such set before and after a revert, and the right panel is the file each set produces.

The alternative is to store patches and replay them, which is what Darc's and Pijul do at the line level [59, 74]. We chose sets over replay because replay introduces ordering: reverting a feature means computing an inverse patch and applying it to the current state, and the result depends on what the current state is. A set-based revert is idempotent (removing what is already absent changes nothing) and its inverse is exact (restoring a removed set reproduces the original bytes with no drift). Those two properties make the preview trustworthy, because the files the preview shows are the files the operation will produce, regardless of what else has been recorded since. The cost is that every function must have exactly one included version at any moment, which is the constraint the fork rule (Section 4.3) enforces.

Because a version is a set, revert, restore, diff, and switch all take a set and return a set, and two properties follow that a developer can check on their own repository. Reverting the price cache and then restoring it produces a file byte-identical to the one that was there before either command ran, because both operations are set arithmetic over the same records and neither reapplies a patch. Each of the four can also be shown in full before it happens, since working out the resulting files takes no more than rebuilding them

from the proposed set: sgt revert "price cache" draws the twelve edits that would remain, names the functions whose text would change, and asks apply this revert? [Y/N]. Run with no terminal attached it prints the same preview and declines, so a revert nobody watched does not happen.

Three requests wrote api.py::handle, and in Figure 3 it carries an edit from the rate limiter, one from the cache, and one from the retry policy, in that order. Removing the cache's edit under the first rule alone means removing everything recorded above it in that function, and the retry policy is above it. We shipped that behaviour first, and on a history where features interleave inside a shared function it removed a later feature in order to remove an earlier one, with a preview that counted the edits coming out and never said which features they belonged to, until it emptied a test repository we were using for exactly this measurement and we changed the default in August 2026. The default now keeps the later edits and subtracts the reverted feature's lines from the current text of the shared function with a three-way merge, the same operation git performs when it merges a text file. The version of api.py::handle that Figure 3 leaves in the file is produced that way, and the hollow diamond beside it is how sgt says so. The old behaviour remains available as revert --take-dependents, and its help text says what it removes.

That three-way merge is the only place sgt changes text that no person wrote, so sgt names every function it changes that way and every function it decided to leave alone. When the subtraction overlaps a line a later edit changed, sgt leaves the function exactly as it is and prints kept unchanged (the removal overlaps later edits—needs your edit) with the function named, and when a function sgt is keeping still calls a function the revert removed, it

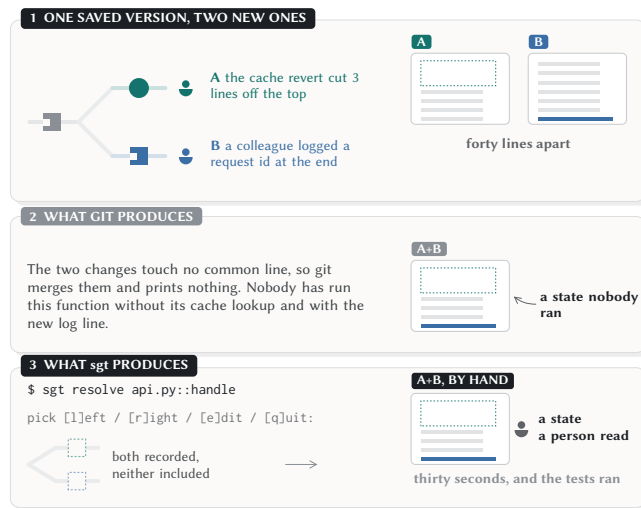


Figure 4: Two edits to one function from the same saved version, forty lines apart. Git produces a state nobody ran and sgt produces a question, and of everything sgt refuses this is the only refusal that can stop work which would have been correct.

prints still references removed code (fix or revert separately). Both are reports and neither stops the revert, because the alternative is a command that refuses to finish over a consequence the developer may already intend. The cost of the default is that after the merge the shared function is text the developer has not read, and their test suite decides whether the revert was correct. Naming the function before the revert runs is as far as we are willing to go on the developer's behalf.

4.3 Two versions of one function, and a person decides

When two edits change the same function from the same starting version, sgt records both, includes neither, and asks the developer. This is the commitment that costs the most goodwill in the first week, so we want to be precise about whose behaviour we are departing from and why theirs is right.

Git merges text and knows nothing about what a function is, and a system that has to work on every text file in every language anyone will ever write can afford to know nothing else, which is why git merge is useful on a Makefile. In the case Figure 4 draws, two people had last saved the same version of `api.py::handle`, one of them reverted a caching feature, which took the cache lookup out of the first three lines, and the other added a request identifier to the logging block forty lines away, with no line in common. Git merges the two without comment, and the function that comes out has never been run in that state by anybody. sgt sees one function with two versions from one parent, names it, and stops. sgt resolve `api.py::handle` prints the two versions beside each other and asks pick [l]eft / [r]ight / [e]dit / [q]uit, where the third choice opens the file in the developer's editor and the fourth leaves the

fork open, and whatever they write has to pass the project's test command before sgt will include it. Where no test command is configured the developer supplies the verdict with `--override pass` and a `--reason`, and the reason and their name are recorded beside the resolution, so the next person to read that function finds out why it looks the way it does.

A developer pays for this by being stopped in cases where the merge would have been fine, and in one of those cases sgt stops for nothing at all, since deleting a function and adding it back with byte-identical content in two places produces two versions from one parent and sgt treats content that agrees exactly as a disagreement. Our position is that the cost of being asked is bounded and visible at the moment it is paid, and the cost of a silent textual merge inside one function is a failure that surfaces later somewhere else. We hold that loosely, because a developer who merges thirty branches a day would reasonably weigh the two the other way, and for them the current design is worse.

4.4 The names are inferred, and correcting one moves labels only

Every figure in this paper labels a group of edits with a name like *price cache*, and those names are the part of sgt that guesses. Functions that changed in the same saves and that refer to one another are grouped together, using the change coupling signal established by Zimmermann et al. and Ying et al. [96, 101] and a community detection method over the resulting graph [87]. A language model then reads the saves in a group and writes the label, which is the descendant of a task attempted before language models existed, when Buse and Weimer generated a description of a change out of the change itself [11]. The label draws from the developer's words rather than from the code's identifier names, because those two vocabularies diverge: Nguyen and Glassman showed that the naming a model uses in its output and the naming a developer uses to describe the same behaviour systematically mismatch [66], so a label derived from identifier names would speak the model's language rather than the developer's, and the developer who later searches for "the caching" would find nothing under whatever the model called its cache class. Membership is computed from the code and the label is written from the words, and those two steps fail in different ways, so we treat them separately.

We considered asking the developer to confirm the grouping at save time and decided against it, because answering that question takes the same knowledge commit hygiene takes, and a developer who could reliably answer it could also have staged the hunks. Shipman and Marshall collected the reasons people abandon systems that ask them to formalise their own work, premature structure among them, and the design they arrived at has the system propose a structure that its user may accept, modify, reject, or leave alone [79]. That is the arrangement here, and we are applying it under easier conditions than the ones they wrote about. The structure their systems tried to elicit was tacit, so a user asked to state it had first to work out what they thought, where the structure sgt recovers was said out loud, in a request, before any of the code existed, and the only reason it needs recovering is that nothing wrote it down. What matters is therefore not whether the guess is right but what a wrong one costs, and correcting one moves labels

and touches no recorded edit. `sgt feature rename 001f73c2 "price cache"` renames a group whose identifier stays what it was, `sgt feature regroup` moves functions between groups, and `sgt save --as "price cache"` files a new save into a group that already exists. Because identifiers survive all three, a revert written down in a code review comment last month still removes the same edits after somebody renames the group today.

Two failures show up often enough that a reader will hit them. A function that everything calls, an argument parser or a logging helper, changes in nearly every save and pulls otherwise unrelated work into one group, and the fix is a regroup the developer has to notice they need. A save with no message gives the language model less to read, and `sgt` says so at the time, printing `· no words` captured under the save it just wrote, because the label summarises what the developer said about the work and a developer who says nothing gets a name derived from the code alone.

4.5 What sgt will not do

`sgt` writes no code of its own. Every byte in the working files came from the developer or their agent, apart from the three-way merge splice in Section 4.2, which reports every case it cannot perform cleanly. We have been asked more than once during this work why `sgt` does not hand the shared function to a language model and have it redrafted after a revert. We considered it seriously and rejected it for one reason: a revert whose output depends on a model's judgement produces a file whose correctness the developer can verify only by reading every line the model touched, which is the labour Section 3 argued this tool should eliminate. A revert whose output is computed from the records the developer already validated produces a file whose correctness follows from the records, and the developer's test suite decides whether the composition is sound. The cost is that a subtraction which overlaps later work requires a hand edit; the benefit is that the developer knows what they are reading.

`sgt` also does not stand between a developer and git. The records are files under `.sgt/` committed alongside the code, `sgt git <args>` passes through to git untouched, a colleague who has never installed `sgt` clones an ordinary repository with an ordinary history, and there is no server and no background process. Two commands refuse to run: `switch` stops when there are unsaved edits, because `sgt` has nowhere to put them, and names the files and prints record them with `'sgt save'`, or `commit / 'git restore'` those files, then `re-run`; `restore` stops when putting a set back would leave two versions of one function live, and there its message names the edits by identifier while the function whose two versions caused the refusal goes unnamed, which is the rough edge Section 5 reports us fixing in the fork warning and not here. Everything else `sgt` finds wrong it reports, and then it proceeds.

4.6 From the five operations to the commands a developer types

Figure 5 sets out the nine operations that act on the record, each with its cost beside it, and the five operations Section 3 ended on are four commands and one thing that needs no command. Recording a boundary without declaring it is `sgt save -m`, run whenever the developer feels like running it, since the grouping is recovered

afterwards from the code. Seeing the work one request produced is `sgt show "rate limiting"`, or `sgt show "rate limiting@1"` for one stretch of work on it alone. Removing a named set is `sgt revert "price cache"`, and carrying one to another branch is `sgt propose land --subset "export"`, which declines a subset that omits a group another chosen group depends on and names the group that is missing. Each of those four takes as its argument a phrase somebody used while the work was happening, so the recovery Section 3 described is eliminated entirely. Discovering the layering needs no command at all wherever the references resolved, because the cache edits record that they were built on `api.py::_bucket` and therefore on the rate limiter, so the developer has at any later moment the fact they needed at 14:30 and did not have. Where they did not resolve the layering is simply absent rather than wrong, and the developer is back to reading the code, which is the position the ambiguity rule above trades down to on purpose. The rest of what a developer types is housekeeping around those nine, `sgt init` to begin, `sgt log` and `sgt sync` to see where things stand, and the `sgt feature rename` of Section 4.4 to correct a name, none of which changes which edits a version includes. Section 5 runs these in order on one repository.

5 One Repository, Five Sitzings

The repository here is a five thousand line Python service that answers price quotes for a trading dashboard, and the first sitting is the hour Figure 1 draws. We assembled the sequence from sessions we ran while building `sgt` so that one repository contains the five operations Section 3 asked for, in that order, followed by the one case where `sgt` stops and asks. Every command below runs and every block of output came out of `sgt` as it is today, with lines too wide for this column wrapped at the `·` separators, with two displays too tall for this column replaced by a description of them in brackets, and otherwise unchanged. The sequence is arranged to show which decisions the developer is left to make and when in the work each one arrives.

5.1 Monday, 14:02 to 15:20: two saves and four names

At 14:33, with the rate limiting finished and the price cache first drafted, the developer saved.

```
$ sgt save -m "rate limiting and a price cache on quotes"
✓ save c39ab2f "rate limiting and a price cache on quotes"
├─o new feature (4a1c9e02) — unnamed; name it:
│   sgt feature rename 4a1c9e02 "<label>"
│   api.py::_bucket, api.py::_key, api.py::_window +1 more
├─o new feature (001f73c2) — unnamed; name it: ...
│   cache.py::get_cached, cache.py::_ttl, db.py::query +1 more
├─o new feature (9d20b18a) — unnamed; name it: ...
│   db.py::fetch_row, api.py::export_row, util.py::load_row +1 more
△ one save touched 3 features — deliberate? `sgt log --map'
  shows them; `sgt feature regroup move' re-files work
↪ reverse this save: sgt undo
```

The developer typed one sentence and `sgt` wrote twelve records. The three groups have no names yet, because `sgt` writes records when the developer saves and works out the names when the developer next reads, and a name costs a language model call that has no business running while somebody is mid-session. The warning is

the recorded set	COMMAND	WHAT THE DEVELOPER GETS	WHAT IT COSTS
WHAT WRITES RECORDS			
	<code>sgt save -m "..."</code>	One record per changed function, filed under a name for the request. The only command that adds to the record.	The grouping is a guess. Correcting it moves labels and changes no recorded edit.
WHAT ONLY READS THEM			
	<code>sgt show api.py::handle</code>	The request this function came from, the saves that touched it, and how many edits removing that request would take out.	The answer is exactly as good as the grouping behind it, and it does not say what the request was built on.
	<code>sgt diff main feat/export</code>	The edits each version has that the other does not, in both directions.	It answers in functions, so seeing which lines moved still means opening the file.
	<code>sgt intent list</code>	Each feature cut into the stretches of work done on it, each with a name like <code>rate limiting@2</code> the other commands accept.	A language model places the cuts by reading the saves, so a stretch can start one save too early.
WHAT CHANGES WHICH RECORDS A VERSION INCLUDES			
	<code>sgt revert "price cache"</code>	The named set subtracted from the version, with every function it changes named before anything is applied.	A function later work also edited gets the set's lines subtracted from its current text, and the tests decide.
	<code>sgt restore "price cache"</code>	The same set back, byte for byte, with everything added since left alone.	Refused when putting the set back would leave two versions of one function live, and the refusal prints edit identifiers with no function name.
	<code>sgt switch feat/export</code>	The working files rebuilt from that branch's set.	Refused on unsaved edits, because sgt has nowhere to put them yet.
	<code>sgt resolve api.py::handle</code>	The two competing versions side by side, and whichever one the developer picks or writes is included once the project's test command passes.	Two edits at opposite ends of one function are also competing versions, and git would have merged them without asking.
	<code>sgt propose land --subset "export"</code>	One finished feature sent to the shared branch, with the unfinished one left behind.	Two functions that sit next to each other in one file can lose the blank line between them.
 an edit the version includes  an edit it does not No argument here is a hash or a line number: each is a request somebody named, a branch, or a function.			

Figure 5: The nine operations that act on the record, grouped by what each one does to it. One command writes records, three only read them, and five change which of them a version includes.

the one line here that resembles what git asks for, and sgt printed it after the save was already written; the developer read it, agreed the sitting had been mixed, and carried on, which is the case Section 3 opened with.

The second save at 15:20 covered the retry policy and a rework of the cache the agent made while adding the retry. By then the first save's groups had names, so its echo listed • Price Cache (001f73c2) for the group that already existed and ◦ new feature (7c02b4d1) for the retry work. `sgt log --tree` afterwards printed the four groups the hour produced.

```
Rate Limiting (4a1c9e02) · 4 symbol(s)
Price Cache (001f73c2) · 4 symbol(s)
Retry Policy (7c02b4d1) · 3 symbol(s)
Row Accessors (9d20b18a) · 5 symbol(s)
```

Three of the four names are what the developer would have written, and the fourth is the agent's unrequested rename, named from the code because the developer never said anything about that part of the work, which is the failure Section 4.4 predicts. It is also the only one of the four whose name nobody needs, so nobody corrected it.

5.2 Thursday: changing the rate limiter

The fixed one minute window has to become a sliding one, and the developer starts from what the rate limiting is now.

```
$ sgt show "Rate Limiting"
```

```
feature 4a1c9e02 "Rate Limiting"
4 edits · 4 symbols in 1 file · last touched 3d ago
symbols api.py::_bucket, api.py::_key, api.py::_window,
api.py::handle
saves c39ab2f rate limiting and a price cache on quotes
reverting this removes 4 edits
```

The four functions are the answer to the question that took four runs of `git log -L` in Section 3, and nothing here required the developer to guess that four was the right number of places to look. What this output does not say is that the price cache calls `api.py::_bucket` and therefore sits on top of the rate limiter, so a sliding window is a change underneath work that already depends on it. The record holds that fact, because the cache's edits name the functions they refer to, and show declines to print it, so the developer rewrites `_bucket` without being told who reads it. `propose land` refuses on this same relation on Wednesday, which is what makes the omission here a decision about what to report and not a gap in the record.

The agent rewrites `api.py::_window` and `api.py::_bucket` and leaves the other two functions alone, and the developer saves, naming the group this time because they already know which one it is.

```
$ sgt save --as "Rate Limiting" -m "sliding window, 60s of buckets"
✓ save 1f0aee3 "sliding window, 60s of buckets"
└─• Rate Limiting (4a1c9e02) api.py::_bucket, api.py::_window
✓ named "Rate Limiting"
```

--as is the one place a developer states a grouping by hand, and it changed nothing here, because both functions were already in that group and the edits would have been filed there without it; they typed it because they knew the answer and had no reason to let sgt work it out. The rate limiting now has two stretches of work on it, and sgt log --focus 4a1c9e02 names both.

```
• Rate Limiting 4a1c9e02 · 2 checkpoint(s)
[0■■■] 4a1c9e0200 :rate-limiting Rate Limiting
[1■■■] 4a1c9e0201 :sliding-window Sliding Window
operate:
sgt revert 4a1c9e02:sliding-window (one checkpoint, by name)
sgt revert 4a1c9e02 (whole feature)
sgt revert 4a1c9e0201 (by index)
```

The developer never asked for these two stretches. sgt cut the work where their words changed, which in this repository is where one request ended and the next began, and either half is addressable on its own, so sgt revert "Rate Limiting:sliding-window" takes Thursday's work off and leaves Monday's four edits in place. The bar beside each name puts that checkpoint's edits in their place in the history, read left to right, with a middle dot where a span of the history holds none of them, so the terminal spends four characters on the encoding Figure 1 spends a panel on. The three forms under operate are printed because people on our own team found that a stretch of work had a name and then stopped, not knowing what would accept it.

5.3 The following Monday: taking the cache out

The cache serves a stale price for up to sixty seconds and has to go.

```
$ sgt revert "Price Cache"
[the log region redrawn, with the 6 removed edits marked]
subtracted from shared code (later work kept): api.py::handle
removed going forward: cache.py::get_cached, cache.py::_ttl,
db.py::query
△ still references removed code (fix or revert separately):
api.py::export_row, util.py::load_row, util.py::save_row
apply this revert? [y/N]
```

The retry policy edited api.py::handle after the cache had edited it, so that function keeps the retry work and loses the cache's three lines, and a request made a week ago comes out from under a change made yesterday. Two functions existed only for the cache and come out with it, which empties cache.py and removes the file, while db.py::query loses the parameter the cache added and goes back to the text it had at 14:02. The three functions in the last line are row accessors the agent rewrote during its rename, and it pointed all three at get_cached while it was in there, so their edits belong to a request the developer is keeping and name a function the revert deletes. sgt names them and proceeds, because whether the repair is deleting three call sites or reverting the rename as well is a question about the developer's intent, which the recorded edits cannot answer.

The developer answered y and ran the test suite, which failed in the three places the report named and nowhere else, and the repair was deleting the cache lookup from each of them. sgt builds every one of these operations as set arithmetic so that a revert can be taken back exactly, and the developer tested that claim before trusting it.

```
$ sgt restore "Price Cache"
✓ restore applied — 0 edit(s) removed, 6 added.
('sgt undo' reverses this.)
```

Putting the cache back returned cache.py, db.py::query, and api.py::handle to the bytes they held before the revert, and it left the three call site repairs the developer had made in between alone, because a restore adds records to the set and rebuilds the files from the result without reapplying a patch. They had wanted the cache gone, so they reverted it a second time, and that run printed the same three consequence lines as the first.

5.4 Wednesday: shipping the export fix alone

The export endpoint from Section 3 has to ship to the release branch today, and the branch it sits on also carries the half finished retry policy. The export code goes into a new export.py and applies the rate limit before streaming, so export.py::write_csv calls api.py::_bucket, and nobody thought about that call as a dependency while writing it.

```
$ sgt propose create --base release
✓ proposal 3f9a2c81b04d (base release, +17 op(s), 4 feature(s))
$ sgt propose land 3f9a2c81b04d --subset "Quote Export"
× 'Quote Export' requires Rate Limiting -- include it in
--subset too
```

The developer asked to move one group and sgt named the group that has to come with it, which is the fourth operation Section 3 asked for. The record of the call to api.py::_bucket was written at the moment the export code was saved, and sgt reads it now to answer a question nobody was thinking about then, where git would have taken the cherry-pick and the release branch would have failed to import.

```
$ sgt propose land 3f9a2c81b04d --subset "Rate Limiting" "Quote
Export"
✓ propose land release: 8b21c0d4e7fa (+8 op(s))
```

Two groups went to the release branch in the order the records put them in, and the half finished retry policy and the accessor rename stayed behind. Those two groups are the stack from Section 3, assembled without the branch that had to exist before the work did.

5.5 Thursday: two versions of one function

A colleague on this repository had been editing api.py::handle as well, adding a request identifier to every outbound log line, and neither of them knew about the other's edit until the developer pulled.

```
$ sgt sync
△ sync origin/main: merged 4f2a1c8b93de
+7 op(s)
△ 1 OPEN FORK(S) — forked symbol(s) sit at the common
ancestor until resolved:
api.py::handle → sgt resolve api.py::handle
```

The remedy is printed as a function name because that is what the developer knows about the situation. For months the fork warning printed sgt advanced merge-op 3c91ab02 8ee14d77 instead, which names the two records in conflict exactly and leaves the developer to work out which function they are in, and it is the clearest instance in this repository of a report that was accurate and useless. The two versions of api.py::handle are the one Monday's revert produced and the colleague's, which changed no line the revert had touched, so this is the case Figure 4 draws, reaching the developer as a consequence of an operation they ran earlier the same week. sgt prints them beside each other and asks.

```

— fork on api.py::handle — (◀ left = tip A ▶ right = tip B)
[the two versions of the function, side by side]
pick [l]eft / [r]ight / [e]dit / [q]uit:

```

The developer picked e, merged the two by hand in about thirty seconds, and sgt ran the project's test command before including the result. We are not going to claim the textual merge would have been wrong here, because it would most likely have been right, and sgt charged thirty seconds and one interruption to establish that. We designed for the case where the merge is wrong, one edit adding an early return and the other edit's new code sitting below it, with the test run that would have caught it happening on a branch nobody has pushed yet. A developer who merges often and reads carefully is paying us for a certainty they already had.

5.6 What this cost

Across five sittings and one fork the developer made three decisions sgt could not make for them. They decided the export endpoint should ship without the retry policy, they decided how the two versions of `api.py::handle` should read, and they decided that the three functions still calling the cache should lose that call and keep the rest of the agent's rename. All three are decisions about the code itself, and none of the three is a question about which lines belong in which commit.

The developer paid for those three decisions in three places. One group of five edits carries a name derived from the code, because nobody said anything about that part of the work; the revert left `api.py::handle` holding text nobody had read, so their test suite decided whether the revert had worked; and the fork stopped them on a merge that was probably fine. The second and third are direct consequences of Section 4.2 and Section 4.3, and on a repository this size we would choose both again.

6 What Would Show This Is Wrong

Three claims in this paper fail independently of one another, and each one fails in a way that a different kind of evidence catches. The record either reproduces the bytes the developer's editor and agent left in the files or it does not, and that is a question about a program, settled by running it over histories that already exist. The grouping either lines up with the requests a developer made or it does not, and that is a question about an inference, settled by comparing it against sessions where somebody happened to write the requests down. A developer either gets more of these operations right with sgt than without it, or does not, and that is a question about people. The first has a procedure and a number. The second is under way on a corpus we are still assembling. The third is designed, validated by three pilots (Section 7), and pre-registered before recruiting. Underneath all three, sgt has been the version control for its own repository since the day it first ran, which is where every number in Section 9 comes from and why the two prices Section 4 states most flatly are the two that cost us something before they cost anything else.

This section declares the study before its data exists. The measure that decides the argument is the errors a participant does not notice, and naming that measure before the data exists is the only way to be believed about it afterwards. A reader who encounters the result in a future paper can check it against the prediction stated here; a

result that appeared without the prediction could have been chosen after the fact to make the design look good. The pilots in Section 7 tested the tasks, the tools, and the rubric; they did not produce the between-subjects comparison this design calls for.

6.1 The rebuild has to be exact

If sgt reassembles a file into bytes that differ from what the developer left there, then every operation in Section 4 is set arithmetic over a fiction, and nothing else in this paper is worth reading. This is the one measurement whose failure would end the work rather than qualify it, so it goes first and it is the only one we insist on before adoption.

The procedure follows the shape of the claim. We walk a repository's history one commit at a time, mine each commit into records, rebuild every file from the set of records that commit's state implies, and compare the result against the blob git holds, byte for byte. A repository passes at a commit only if every file matches, so a single differing byte in a single file is a failure at that commit, and we will report the fraction of files that rebuild rather than an average over repositories, because a developer cares whether their file comes back and not whether most files do. The corpus is open source Python and TypeScript repositories chosen for length of history and nothing else, and we are still assembling it, so the only number this procedure has produced comes from our own repository, which is the hardest case available to us because records written by five different versions of our miner sit in it.

That number is fifty-six files that no valid set of records reproduces, one in seven of the files whose records name functions, and Section 9.3 gives that denominator and says what is in the fifty-six. What we do not know is whether fifty-six is the cost of our own format changes or the cost of the design, and since five miner versions in fifty-seven days is a rate no adopter will ever match, the corpus run is how we intend to separate the two: a repository sgt has never touched cannot have our migrations in it.

6.2 Whether the grouping matches what was asked for

The names in every figure in this paper are inferred, and a developer who types `sgt revert "price cache"` is trusting that inference with the contents of their files. The untangling literature evaluates clustering against a partition somebody drew, for the good reason that its authors had nothing better to compare against (Section 2). We ask a different question: whether the set that comes back when a developer names one request is the set of edits that request produced.

That question needs a corpus where the requests were recorded, which agent sessions now supply as a matter of course, since the transcript of a session names each request and the files the agent touched while carrying it out. We mine each repository without reading any transcript, derive a reference grouping from the transcripts alone, and then ask two things of every request in the session: how much of what that request produced comes back when the developer names it, and how much else comes with it. Those are the two ways a revert disappoints the person who ran it, the edit it missed and left sitting in the file, and the edit belonging to a different request that it carried off as well. They fail asymmetrically,

and the second is worse, because a developer who is handed too little notices when the code still runs the way it did and a developer who is handed too much has already lost work they meant to keep.

We expect the shared helper to be where this measurement goes wrong, because Section 4.4 predicts it and fifty-seven days of daily use have not stopped it happening. An argument parser or a logging function that changes in nearly every save is coupled to everything by the signal the clustering reads, and it draws otherwise unrelated requests into one group. If most of the disagreement between our grouping and the transcripts turns out to sit on functions of that kind, then the fix is to weight a function's coupling down as the number of things it couples to grows, and that is a change to one term rather than to the design. If the disagreement is spread evenly across ordinary functions instead, then the coupling signal is not carrying the grouping and the developer should be asked at save time, which is the design we argued against in Section 4.4 and would have to argue for.

6.3 Three requests, a transcript, and code nobody in the room wrote

The study puts a developer in the position the paper is about, which means giving them a repository whose code they did not write and the three sentences that caused it. Each participant receives a five thousand line Python service, a transcript of the session that produced its most recent hour of work, and three tasks, each of which names one request from that transcript and asks for it to be moved or removed without disturbing the other two. This is the state of the developer in Section 3 a week later, and it is also the state of the developer who picks up a colleague's branch on a Monday, and it is the only arrangement we found in which every participant knows exactly as much about the code as every other one. Participants who direct the agent themselves each remember a different amount of what came back, and that remembering is the thing whose absence we are claiming matters.

We recruit twelve developers who use a coding agent as part of their ordinary work, because the difficulty we describe belongs to them and to nobody else, and we ask each of them to work through two repositories, one with git and one with sgt, in a counterbalanced within-subjects order (four groups of three, fully crossing condition order with project assignment). Both conditions include the same AI coding assistant and the same editor, differing only in how history is recorded and queried; the git condition includes GitLens in the editor, because comparing a graphical extension against a bare terminal would measure the presence of a view rather than the representation underneath it. Each repository holds three requests whose edits interleave inside shared functions, and each has a different set of three so that a participant who has learned the shape of the first task cannot apply it to the second. The three requests are: identify what a tangled commit introduced and when (provenance, five minutes, three closed questions scored against a key); remove one named feature without breaking the other two (entangled removal, fifteen minutes shared with the third request); and correct a dependency that the removal broke (a repair under time pressure that tests whether the participant noticed the col-lateral). With twelve participants we can detect large effects and

nothing else, and the paper will say so in its limitations rather than obscure the sample size behind a p -value.

Request 1 is scored by key: three closed questions, each with one correct answer and an "I could not tell" option that is real data rather than a wrong guess. Requests 2 and 3 are scored by script, and the script asks three questions. The tests belonging to the removed request fail, the tests belonging to the two kept requests pass, and no function outside the intended set differs from the reference repository. That third check catches the failure this paper is about, because a participant who removes the waitlist and quietly takes half the enrollment logic with it has produced a repository that passes its own tests.

The first row of Table 1 is the one that matters, because a developer who spends an extra twenty minutes untangling a commit by hand has lost twenty minutes, while a developer who submits a repository they believe is clean and that still calls a function they deleted has lost something they will not find until the release branch fails to import. Hand untangling fails silently, and a design whose whole argument is that the developer no longer holds the decomposition has to be measured against silent failure rather than against the clock. We collect time because a reader would want it and because a tool that is correct and unbearably slow is not adopted, and we would not publish a time difference as the finding.

The last two rows of Table 1 measure the prices we stated in Section 4 rather than the benefits, and they are there because a study that measures only what a design buys is a study that cannot be wrong. A fork stops the developer and a wrong label makes them type a rename, both of which are countable inside the session, and both have a threshold past which we would change the design rather than defend it.

Three pilots (Section 7) validated this design before the full study: one in each condition tested the tasks, and a third exercised the study apparatus end to end. Both task conditions completed all six requests, confirming that the primary measure is error rate and cost rather than task completion. The pilots found twenty-one tool defects that would each have cost a real participant their data point, four defects in the study design that would have made a result uninterpretable, and one validity threat (a confound favouring our condition) that we disclose in Section 7.4. The tool participants receive is a different version from the one the pilots ran.

6.4 What none of this would tell us

Only the first of the three claims can be pushed past by running more of the same, since adding repositories to Section 6.1 sharpens that number and nothing else here improves with volume.

A recorded request is a sentence, and a sentence is less than an intention. A developer who types *make the price lookup faster* holds a view of what should change that the sentence does not contain, and the agent's response is a guess at the remainder. The alignment we measure against a transcript is therefore alignment with what was said, and the transcript sets a ceiling on it that better clustering alone cannot exceed. We think this is the right measurement anyway, on the ground that what was said is what the developer will type when they come back to remove it, but a reader who believes intention and utterance come apart more

Table 1: The six measures of the study in Section 6.3, each with the result that would count against the design. The first row is the one we would report first, and the third row is the one a reader would expect us to report first.

What we measure	What we expect	What would count against us
Submissions the participant believed correct and that were not.	The gap here is wider than the gap in time. Removing a request touches code the participant never read, and in git nothing tells them so.	Equal rates in both conditions. The recovery we call expensive would then be something developers already do reliably, and Section 3 is a story about us.
Submissions that pass the check.	More of them with sgt, and the ones that fail fail in the places revert named out loud.	A failure in sgt that the preview did not name. A report that misses a consequence is worse than no report.
Time to the moment the participant says they are done.	Shorter with sgt, and we would not publish this alone.	Shorter with sgt while the first row is flat. sgt would then be a convenience, and we have argued for it as something else.
Lines of diff the participant opened before submitting.	Far fewer with sgt, since no command in Section 4 asks anyone to look at a hunk.	Participants reading the diff anyway. They would be checking sgt's report rather than using it, and after twenty minutes of that they are doing the git task with extra steps.
Forks sgt raised, and how many the participant judged pointless.	A small number, mostly judged pointless, all cheap.	Any participant switching the rule off, or a fork resolved by picking a side whose result then failed the check. Being stopped would have bought nothing.
Labels the participant corrected, and what the correction cost.	One or two renames, no regroups.	A regroup on the critical path of an operation. The inference would then be spending the developer's attention rather than saving it.

often than we do should read that number as an upper bound on something smaller.

Two hours with a transcript understates a week of forgetting. The cost we describe in Section 3 grows as a developer's memory of the session fades, and our design substitutes reading somebody else's transcript for that fading. A participant who has just read three sentences knows more about the session than the developer who typed them a week ago, so the git condition in our study is easier than the situation it stands for, and the difference we measure understates the one that matters. We prefer the understatement to a study nobody can run.

Finally, every task in the study has an answer a script can check, and we chose those three tasks for that reason. The operations a developer wants most may be the ones with no checkable answer, and deciding whether a stack of two reviews was the right shape is one of them. We can say what sgt makes possible there and we cannot measure it, and Section 9 keeps that distinction.

7 Pilot Study

We ran three pilots before the full study: one in each condition (sgt and git, both on the same task set) and a third exercising the study apparatus end to end. The first two were designed to find defects in the tool that would cost a real participant their data point, and defects in the study design that would make a result uninterpretable. The third tested the machinery between the participant's machine and the facilitator's console—exactly the seam nobody had exercised—and found a validity threat that neither task pilot could surface. We report all three because the defects they found are consequences of design commitments rather than accidents of implementation: a technical evaluation establishes that a property holds across 10,237 operations; it takes a new user to show that a property that holds still costs too much to use.

Request	sgt condition	git condition
R1: Provenance	Correct, 4 cmds	Correct, 6 cmds
R2: Removal	Done; tool corrupted file, repaired by hand	Done cleanly
R3: Selective restore	restore crashed; re-covered via git	Done cleanly
R4: Regression repair	Correct fix + better test	Correct fix + better test
R5: Plan & fork	Done in git (declined sgt)	Done cleanly
R6: History edit	Half done (no rename verb)	Done (scripted re-base)

Table 2: Per-request outcomes across both pilot conditions on coursecraft (same tasks, same rubric). The sgt condition ran on version 0.1.0; twelve defects fixed afterwards. R4 and R5 produced equivalent results in both conditions.

7.1 What the pilots tested

Six requests, drawn from the scenarios of Section 3: find which commit introduced two unrelated changes (provenance), remove a named feature while keeping the rest (removal), selectively restore a single function (selective restore), locate and repair a regression whose test still passes (regression repair), plan work on a branch and resolve a conflict (plan and fork), and split a tangled commit so each half has a clear name (history edit). Both conditions worked on the same five-thousand-line Python project and were scored with the same rubric and the same automated check. Table 2 summarises the outcomes.

7.2 What the sgt condition found

The sgt pilot completed four of six tasks and reached the correct answer on a fifth. Three of the eight verbs in the daily interface failed during the session: revert produced syntactically invalid Python, restore crashed with a raw traceback, and save refused to record a legitimate edit. The participant fell back to git for the repairs and finished the substantive work outside the tool.

Twelve defects were fixed from this one session. We describe three because they are consequences of design commitments this paper defends.

Semantic subtraction glued lines together. Reverting the waitlist feature left one file with three function headers on one line, find_student relocated to end of file, and thirteen collection errors in pytest—under a green checkmark. The cause is the round-trip design of Section 4.2: the rebuild synthesises zero separator bytes, so a removal that takes the layout chain of an entity it keeps produces definitions with no whitespace between them. The fix re-grounds the layout of every kept entity after a subtraction.

A preview flag mutated state. The tutorial teaches that every destructive verb shows a preview first and acts only with --yes. Adding --keep-dependents to a revert silently voided that contract, writing intermediate state into the store with no preview and no confirmation. The participant's verdict: "If a flag can turn a preview into an action, I can't reason about which of these commands are safe." The flag now requires --yes.

The map showed husk features. sgt log --map displayed a feature called *Section Waitlist* that contained zero symbols in zero files—a cluster whose only records were bookkeeping sentinels—while the feature that actually held the waitlist code was named after a deleted experiment and did not appear in the view at all. The participant followed the wrong name for the entire removal task. Later validation found the problem was systemic: 9 of 24 leaf features in each study repository owned no behavioural code, including the three features the study's own removal requests name. The mechanism was that the clustering graph includes positional gap-bytes as nodes (they carry within-file cohesion), and Leiden readily forms a community made entirely of them. The fix absorbs any leaf whose members are all non-behavioural into the leaf that already owns the entities those members hang off, reducing coursecraft from 24 leaves to 14 with none empty.

The comprehension operations (naming what a request produced, seeing what a revert would remove) held up under a user who had never seen the tool before; the participant credited "sgt's attribution and its revert preview, which git genuinely cannot do." The mutation operations did not survive contact, and the data-loss path that one of them opened—a fulfill command that overwrote uncommitted work in five files while printing a green checkmark—was the worst defect we found in the system during this evaluation. The participant's conclusion: "From here I treat sgt revert as 'a fast, accurate first pass that leaves me a punch list', not as an operation that completes." When asked at exit whether they would adopt the tool: "For reading history, yes, starting today. For anything that writes, no."

7.3 What the git condition found

The git pilot completed all six tasks inside the time budgets, including request 6 (history edit), which the protocol had expected to be out of reach in the git condition. The participant scripted the interactive rebase, split the tangled commit in two, and verified that both halves pass independently by comparing tree hashes. That result is better than the sgt condition managed on the same task: sgt had already split the same commit at mining time, which is the more impressive half, but the participant could not give the two halves clear names because no verb renames a checkpoint.

Four defects in the study design surfaced:

- (1) The git-condition copies carried commits written by sgt (sgt revert f-10462e1702, sgt undo: restore prior ideal). A participant in the baseline should not be able to tell that another tool was involved.
- (2) A request ticket described a change as "showing times differently" when no earlier format existed. Both pilots spent time on our wording rather than on the task.
- (3) Request 6 needs a rubric for both conditions rather than an assumption that the git condition cannot finish.
- (4) One ticket referenced a duplicate commit and an undo authored by the researcher.

All four are fixed.

The git participant also described, unprompted, what they wished the history could answer: "Ask history a question about a symbol, not a file. 'What happened to overlaps, who calls it, and when did that change?' would have turned request 4 from a hunch into a lookup." A baseline participant independently describing the treatment condition's premise is evidence that the problem this paper addresses exists independently of the tool we built. The need exists before the solution is shown, and we have added a question to the exit interview to collect this evidence deliberately.

7.4 What the apparatus pilot found

The third pilot ran the full study end to end—real bundles, real terminal, real git and sgt—with the sole purpose of exercising the machinery between the participant's machine and the facilitator's console. It found nine defects, all in the seam between systems. Three generalise.

The primary read verb rejected its own output. sgt show failed 6 of the 10 times it was used, and it is the verb the practice sheet tells participants to lean on. It rejected save identifiers printed by sgt log, full git commit hashes, and feature names typed exactly as displayed. The cause was narrower than "identifiers are inconsistent": the id column of sgt log prints the 7-character git commit hash, and show did not accept that format as a selection kind. The participant fell back to git blame and git show --stat—in the sgt condition—and answered the question that way. A tool that pushes someone back to the baseline mid-task is not a measurement problem; it is the measurement. The defect is fixed: show now resolves commit-shaped tokens as saves.

A validity threat: the sgt condition degrades its own baseline. Every commit in the sgt-condition repositories carries its operation identifiers as commit-message trailers. On a representative commit whose actual message is the single line add course search,

we measured 174 Sgt-Op: trailer lines. So `git show`, `git log`, and `git log -p` all bury the real content under a screenful of hex. The git-condition repositories are stripped of these by design, so a participant in the baseline sees clean history while a participant in the treatment sees this. The direction of the bias is the problem: it makes git look worse exactly where sgt is being measured against it, and it does so in the treatment arm—the arm whose participant reaches for `git blame` as a fallback whenever sgt does not have the answer. We disclose this because it favours our condition: a win for sgt on any task where the participant consulted git history is weaker than it appears, and we cannot retrospectively subtract the effect. The pre-registered analysis acknowledges it as a confound.

Carry-over between isomorphic tasks. The two test repositories are isomorphic by design (same episode script, nouns swapped) so nobody sees the same project twice. The pilot participant reported that the resemblance is strong: “by half two I was pattern-matching against an already-solved puzzle rather than reading fresh.” The design counterbalances order and models it as a factor; this finding confirms that modelling order is essential rather than a formality, and that the carry-over effect is likely large relative to the condition effect on a twelve-person sample.

7.5 Thematic analysis

We coded all three pilots’ observations against the design commitments of Section 4, interpreting each through three lenses from the automation trust literature: Parasuraman et al.’s four-stage model of automation [68] (which stage failed?), Lee and See’s trust calibration framework [55] (what did the failure teach the user about the system’s reliability?), and Norman’s gulfs of evaluation and execution [67] (could the user perceive the state, and could they act on it?). Six themes emerged that generalise past the individual defects.

Comprehension held; mutation did not. The comprehension operations—attribution, revert preview, dependency display, feature listing—produced correct answers on first contact in the task pilot. The sgt participant reached the right answer on request 1 in four commands, using a path the git participant could not take (symbol-level attribution across files). Every write operation—revert, restore, save, fulfill—either failed, crashed, corrupted output, or refused to run. The apparatus pilot qualifies this split: the *interface* to the comprehension layer failed at the same rate as the mutation layer (show rejected 6 of 10 queries), so a user who could not navigate to the right screen never reached the comprehension that screen provides. The distinction matters because it separates two failure modes with different fixes: the underlying analysis was correct (the attribution existed and could be displayed), but the path to it was blocked by an identifier mismatch between two surfaces of the same tool.

Norman calls these two directions the gulf of evaluation and the gulf of execution: the first asks whether a user can tell what state the system is in, the second whether they can act to change it [67]. Parasuraman, Sheridan, and Wickens decompose automation into four stages—information acquisition, information analysis, decision selection, and action implementation—and argue that automating the earlier stages while leaving the later ones to the human is both safer

and more robust than automating uniformly across all four [68]. sgt automates stages one and two heavily (parsing, clustering, dependency tracking, consequence display) and leaves stage three to the developer (which feature to revert, how to resolve a fork). The pilot failure sits squarely at stage four: the action implementation that follows a correct analysis either crashed or corrupted output. The design implication is that the comprehension layer is independently valuable even when the action layer is not yet trustworthy, but only if the developer can reach it—an interface failure at the front door costs the same as a corruption behind it, because in both cases the developer falls back to git.

Silent wrong answers erode trust faster than crashes. The participant encountered both an omission error (restore: raw trace-back, no data loss, the system failed to act) and a commission error (revert: green checkmark, three function headers glued onto one line, the system acted wrongly while reporting success). Mishler and Chen showed that commission errors erode trust in automation faster than omission errors, because a system that does the wrong thing while claiming to have done the right thing attacks the operator’s ability to distinguish reliable reports from unreliable ones [62]. That asymmetry appeared here in full: the crash cost one command; the corruption cost twenty minutes of repair work, left the participant unwilling to trust checkmarks, and propagated forward: when the session feature later reported “86 new ops” on a workspace with zero edits, the participant declined to use it. The number was explainable, but the tool had spent its credibility on an earlier false positive. Lee and See’s model predicts exactly this: a crash provides correct calibration information (the system failed and said so), whereas a false positive undermines the calibration mechanism itself, because the user can no longer distinguish a true positive from the next false one [55]. Wang et al. found the same asymmetry in developers using AI code generation: one bad recommendation erases the credibility of three good ones, and a single negative experience with a category of output causes developers to stop reading that category entirely [94]. Madhavan and Wiegmann’s integrative review explains why the recovery never came: people hold automation to a higher initial standard than they hold humans, and a single failure triggers categorical attribution—the system is *broken*—with no intermediate state between working perfectly and being untrustworthy [58]. Sabouri et al. measured the downstream consequence: developers retained only 52% of initially accepted AI suggestions on revisitation, and the retention rate dropped further after encountering a single incorrect suggestion they had initially accepted [75]. Trust is not per-operation but cumulative, and one self-reporting failure costs more than the task it belongs to. Seven such failures appeared across the full evaluation period, three of which we had not found after eight months of daily use and found only by auditing the evaluation’s own results.

The tool’s value was in the punch list. The participant reframed the revert as “a fast, accurate first pass that leaves me a punch list” rather than an operation that completes on its own. Every item on the removal’s consequence report came true: the three functions that still referenced removed code broke, and nothing else did. The preview was the product; the execution was the problem. The participant’s exit debrief sharpened this: “It indexes your history by function instead of by file, so you can ask ‘what changed this

symbol, and what else came along for the ride' and get a real answer in one command—that part is excellent. Use it to find things, then make the change yourself with git." Ferdowsi et al. found that continuously displaying runtime values mitigates both over-reliance and under-reliance on AI-generated code, by lowering the cost of validation through execution [26]. The preview serves the same function: it lowers the cost of evaluating a revert by showing the consequence before it happens, and the pilot confirms both halves of Ferdowsi's finding. Where the preview was correct (the three named functions broke, nothing else did), the participant relied on the tool appropriately. Where it was incorrect (the green checkmark over corruption), the lowered validation cost produced worse outcomes than no tool at all, because the participant trusted the preview and did not verify independently. A partially-working tool can deliver value through its read layer alone—it tells the developer what to change, and they change it with the tools they already trust—provided the reports are accurate. The accuracy condition circles back to the previous theme: a tool whose punch lists are wrong on one task stops being consulted on the next.

Vocabulary mismatch across surfaces. Four counting units—edits, ops, symbols, saves—appeared in different outputs with no definitions, and the participant stopped reading the numbers halfway through: "the edit counts have been meaningless all session." Two views of the same feature reported different symbol counts (the tree counted cluster members; show counted footprint symbols). Selector namespaces varied per verb, with no cross-lookup. Each of these is a consistency defect, not a missing feature, and each one makes the tool harder to learn because the participant cannot build a stable model of what the numbers mean. The fix was mechanical: one counting unit (symbols) and one namespace across all verbs. Carroll and Rosson named the mechanism that makes this costly: active users learn by generalising from one interaction to the next, and inconsistent surfaces punish generalisation by teaching a rule that fails on the second screen [13]. Jackson frames the same requirement as a property of the concepts a system exposes: a concept whose state looks different depending on which operation queries it has no stable operational principle [45]. The problem compounds in a tool whose labels are already inferred: Nguyen and Glassman showed that vocabulary mismatch between a developer and a model is the norm rather than the exception [66], so a developer already arrives at the tool uncertain of the code's naming; a tool that then disagrees with itself across its own screens removes the one stable reference point (the tool's own output) that could have resolved that uncertainty. The lesson is that internal consistency is a learnability precondition (not merely cosmetic), and a tool whose surfaces disagree teaches the user to ignore its numbers.

Baseline sufficiency reframes the study. The git condition completed every task, including the one we had expected to be out of reach (splitting a tangled commit via scripted interactive rebase). The study therefore measures cost and confidence rather than raw capability: both conditions can finish, and the question is how many commands, how many errors, and how much the developer trusts the result. That is a stronger design than one where the baseline cannot finish, because it makes both directions of the comparison

informative—the git condition can win on tasks where the tool's mutation failures cost more time than the workaround saves.

The treatment degrades its own fallback. The apparatus pilot found that the sgt condition makes plain git—the tool a participant reaches for when sgt fails—measurably worse than in the git condition. Every sgt-authored commit carries operation identifiers as commit-message trailers (174 lines on a one-line commit), so git show and git log bury the content a developer needs under hex they cannot use. The git-condition repositories are stripped of these by design, so the baseline sees clean history while the treatment does not. This creates a confound that favours our condition: any comprehension task where the participant consulted git history in the sgt arm was harder than the identical task in the git arm, for a reason unrelated to sgt's attribution. The measurement lesson is that a tool layered on top of a host system (here, git) must account for how it changes that host's usability, because a participant evaluating the treatment is also evaluating a degraded version of the control. We acknowledge this as a threat and note that the pre-registered analysis cannot correct for it post hoc.

The six themes, read together, describe a tool whose analytical core is sound and whose interface to the developer is not yet stable enough to carry the analysis through to action. The first three themes (comprehension/mutation split, trust erosion, punch-list reframing) all resolve in the same direction: the developer can use this tool as a comprehension instrument today, and should not trust it as an execution instrument yet. The fourth (vocabulary mismatch) explains why even the comprehension layer requires polish before study: a developer who cannot navigate to the right screen never reaches the analysis that screen provides, and inconsistency between screens teaches them to stop reading. The fifth (baseline sufficiency) constrains the study design: both conditions finish, so the measurement must be about confidence and error rate rather than completion. The sixth (treatment degrades control) constrains the interpretation: any advantage we observe in the treatment arm is weakened by a confound we cannot remove.

The twenty-one fixes and four design corrections together mean that the study we will run differs substantially from the study we would have run two weeks earlier. Three consequences follow for its design. First, the primary dependent variable is now time and error count per task rather than task completion, since both conditions finish. Second, the exit interview asks whether the participant trusts the tool's reports, because the pilot showed that credibility, once lost to a false positive, does not recover within a session. Third, the sgt condition's tasks are scored on the comprehension half (did the participant identify the right target) separately from the mutation half (did the tool execute correctly), so that a mutation failure in the tool does not mask a comprehension success in the participant. The result that would count against the design is error-rate parity between the two conditions. If both conditions produce equal rates of undetected errors, then the recovery this paper describes is something developers already do reliably without the tool, and Section 3 overstates the difficulty. We report the pilots rather than only the findings, because we want a reader to know which version of the tool the full study runs against and why the study measures what it measures.

7.6 Study repository validation

We built two 5,000-line Python projects (coursecraft and confplan) using scripted agent sessions—28 and 26 commits, 23 episodes each—so that the ground truth for every task is known from the build log. Running the derivability census on both repositories before the pilot produced one pass and one failure: confplan’s central removal task (“take the promotion system out”) was not answerable from the record, because the promotion subsystem sat inside a 40-symbol feature that also held the entire CLI. The same episode script, the same 23 episodes, the same task—answerable on one repository and not the other.

Tracing the cause exposed a single defect with four visible consequences: the largest feature was named *init repo* (a label derived from one seed-commit vote in a 71-op leaf), 77–84% of commits had edits filed under that seed feature, eight features carried file-path labels, and confplan’s promotion subsystem was absorbed into one cluster. The mechanism was provenance blindness on the save path: sgt’s two clustering signals read each operation’s provenance field to learn “these symbols were written in the same sitting,” and every operation saved through sgt save carried no provenance—the commit reference lives in trailers that three call sites did not resolve. 366 of 370 operations, unattributed. Co-change without temporal separation is co-occurrence, which produces one cluster.

The finding that generalises beyond our system: the blindness appeared only in histories built *through* sgt. Every unit test, every dogfood run on the development repository, and every retroactively-mined fixture had provenance, because all of those derive their temporal signal from git commits that already exist. The save path—the one workflow we ask participants to use—was the least-tested path in the codebase, and a tool evaluated only against mined history has been evaluated on its strongest arm.

After three fixes—provenance resolution, label prompt filtering, and an aliasing-rename crash—both repositories pass an 8-of-8 derivability gate: the answer to each of the study’s four requests is reachable through the taught verb set (log, show) in at most three commands. We score this as “8/8 by the author’s reading”: the pre-registered primary metric could not be computed as written, so a referee is entitled to treat the replacement as weaker evidence.

Four tangle cases—commits the build log placed two unrelated changes into—are all separated correctly: in coursecraft, `cli.py::cmd_search` lands in one feature while `slots.py::parse_slot` and its test land in another, and the two concerns the commit message conflates are two features in the record. Reconstruction is exact on both repositories (0 drifted paths), so nothing above came at the cost of fidelity.

One asymmetry remains: coursecraft’s waitlist feature contains five enrollment symbols (a hub function’s edit chain pulls them in), so a feature-level revert would violate the request to “keep enrolling working.” The member list is visible before applying, which makes this a legibility cost rather than a silent failure, and it is the cost we report to participants in the task handout rather than one we hide. On mirrored histories the clustering diverges—the same episode script produces 2 separable features in coursecraft and 3 in confplan—which is itself a finding about the method’s stability under near-identical inputs.

8 What the Design Bought and What It Cost

The evaluation, the corpus, and the pilot each show a different aspect of the same design. We organise this section around what generalises past our system, because an honest account of one tool’s failures is useful to a reader only when it explains what would fail differently had the tool been designed differently.

8.1 Three distinct safety properties

The evaluation surfaced three properties a developer needs from a tool that rewrites files on their behalf, and they require different kinds of guarantee.

- (1) **Nothing is removed.** The store is append-only; every prior state is reachable by identifier. The append-only store guarantees this structurally, and all 10,237 operations in the harness preserved it.
- (2) **The tool operates on what you named.** The revert removes the feature the developer pointed at, the restore brings back the thing they asked for, and neither touches anything else. Three findings in the robustness harness broke this property: an undo that silently dropped a *different* edit, a revert that printed success about an operation it had not performed, and a restore that reported “changed nothing” while emptying a file. All were recoverable—property 1 held throughout—and all were wrong in the direction hardest to detect, because the command’s own report confirmed what the developer expected.
- (3) **New work can still be recorded.** A revert that succeeds must leave the repository in a state where the next save is accepted. One sequence in the robustness harness reached a state where every write verb was refused, including undo, because a layout record outlived the entity it described and the rebuild’s composed image disagreed with the file by one byte. The developer’s work was intact and unreachable.

Property 1 is architectural and held universally. Properties 2 and 3 are behavioural and failed on sequences involving layout records (the gap text and ordering facts that sit between functions in a file). The append-only store makes durability cheap, so legibility becomes the remaining problem. In every one of seven self-reporting failures we found during dogfooding, the data was present and the tool’s account of how to reach it was wrong. Any system that maintains a semantic record alongside a repository will face this same separation, and the lesson is that testing durability (can the bytes be recovered?) answers a different question from testing agreement (did the tool do what it said?). Agreement is harder to test, because the only oracle that can verify it is a probe that independently checks the outcome. The tool’s own report is the thing that failed.

8.2 What each design commitment cost

We trace each of the four commitments of Section 4 back to its evaluation outcome to show where the design has already been tested and where it has not.

The function as unit (Section 4.1). Delivered entity coverage over 42% of files, with a threshold that loses function identity on roughly half of all renames that co-occur with a body rewrite (Section 9.2).

On the corpus the decomposed rebuild rate is 0.23, meaning that three functions in four are absent from the composition at HEAD. Ninety percent of those absences come from the fork rule withholding records, and the parsing itself succeeds, so most of the cost at this level actually originates from the fork rule (the next commitment) rather than from the function-as-unit decision itself. The implication for a designer choosing a grain: the function works as a recording unit—parsing succeeds, identity tracking is imperfect but bounded, and the coverage spans every file tree-sitter can read—but the cost of acting on those records depends on what other commitments the design makes downstream.

Set-based versions (Section 4.2). Gave the round-trip reversibility the walkthrough demonstrates, which the pilot participant called “a fast, accurate first pass that leaves me a punch list.” That reversibility breaks twice in 10,237 operations, both on layout targets no developer would name. The robustness harness found 340 operations that left the repository in an unexpected state, and 68% of those targeted an entity the developer would name—but the dominant violation in those cases (157 of 232) was a layout record being dragged along by the entity operation, while the entity logic itself worked correctly. The set formalism fails in two distinct ways. First, layout records: layout facts are individually addressable because they are symbols, and individually revertible because they are records, but no mechanism keeps them consistent with the entities they sit between. Nine distinct defects trace to this decision, and every robustness-harness failure entered through it. Second, upward closure: reverting a mid-history edit originally orphaned every later edit that depended on it, because the closure rule excluded them from the rebuilt set. On the study repository this demolished 13 files when the participant asked to remove one feature—the worst single defect we found. The fix replaced blanket exclusion with forward subtraction (splice the removed content out of later edits rather than dropping them), which preserves the set semantics while changing what “remove” means from “exclude this and everything above it” to “subtract this contribution from the tip.” The lesson for a designer using set-based versions: closure is correct for the append-only store (nothing is lost) but too aggressive for the working tree (a developer removing one feature does not intend to remove everything built on top of it).

The fork rule (Section 4.3). Protected 162 actually-forked functions and blocked mutation on 27 of 28 outside repositories. The amplification factor is 18: each forked function withdraws records covering 17.7 function keys on average, and the total withdrawal of 2,864 keys is what blocks the write guards. On the self-hosted repository, switching the rule off raises the rebuild rate from 0.44 to 0.88. On the corpus, the median rebuild of 0.33 means the guard requires 1.00 and most repositories are at one third of that. The rule’s read benefit is real (the pilot participant was stopped before two conflicting edits could be silently merged), but the write cost is eighteen times larger than the rule’s description suggests. Nelson et al. found that the expensive part of a merge conflict is the semantic investigation required to understand whether both sides can coexist, not the textual resolution [65]. The fork rule moves that investigation from the moment the merge fails to the moment the second edit arrives, when the developer still holds both intents. Kazemitabaar et al. showed that cognitive engagement techniques—friction that stops

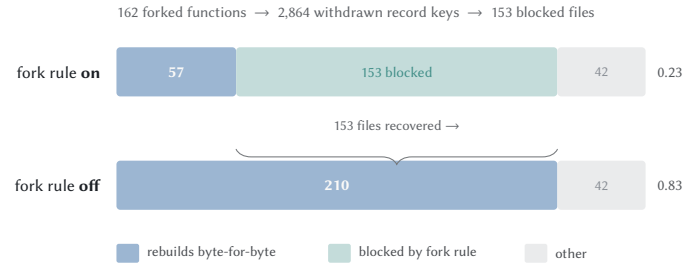


Figure 6: The fork rule’s cost on sgt’s own repository, measured over the 252 files sgt decomposes into functions. With the rule, 57 files rebuild; without it, 210 do. The 153 files the rule blocks are the transitive cost of 162 forked functions withdrawing 2,864 record keys through their dependency chains. The 42 files that fail in both cases are the residual losses of Section 9.3: missing separators, incomplete chains, and the one file whose pending record our own measurement apparatus appended. Whole-file records (104 files, not shown) rebuild at 0.97 in both conditions and move by exactly one file.

passive acceptance of AI-generated code—improve comprehension at the cost of goodwill [48]. The fork rule is exactly this tradeoff: it stops the developer from accepting a merge they have not inspected, and the pilot showed it costs goodwill (the participant declined to use the session feature afterwards). Duan et al. found that distrust propagates between teammates more readily than trust does: one person’s negative experience, relayed by word of mouth, reduces a colleague’s reliance even when the colleague’s own experience has been positive [24]. The fork rule has been tested only with one person, and this contagion effect suggests that a single developer’s frustration with it could suppress adoption across a team even if the rule’s safety guarantee would have helped the others. Figure 6 shows the amplification visually: the 153 files the rule blocks come from just 162 forked functions, and switching the rule off recovers them but removes the safety guarantee.

Inferred names (Section 4.4). Recovered cross-file groupings that git cannot express, and systematically over-decomposed what a developer asked for by a factor of 2 to 6 at the median (up to 13 on the longest-session repository). On CodeNav (the one repository where the metric discriminates), F1 is 0.66 with precision 0.68 and recall 0.64. The margin over a file-only baseline comes entirely from cross-file grouping. The tool’s vocabulary is finer than the developer’s vocabulary, which means the operations of Section 4 require naming something smaller than what was originally asked for. The pilot participant saw five unnamed features for one coherent piece of work and asked why the tool was decomposing what they had done in one sitting. On the study repositories, the feature surface is a reliable index into history (every request’s answer is reachable in at most three commands) but an unreliable decomposition of it. Fifteen percent of labels describe a minority of their members’ actual content, and reverting a 5-symbol feature removes 13 edits, because the structural cut crosses the episode boundary whenever a hub function participates. Study preparation found that

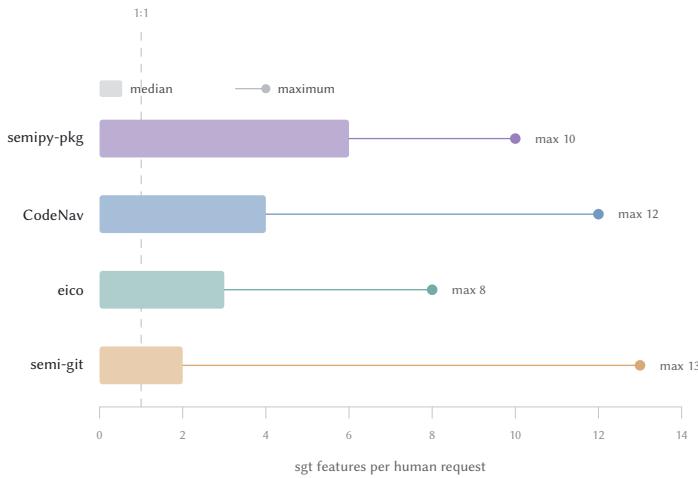


Figure 7: One human request maps to a median of 2–6 sgt features across four repositories with full transcripts. The bar shows the median; the whisker extends to the maximum observed. A ratio of 1 would mean each request lands in exactly one feature. The consistently above-1 ratio means the operations of Section 4 require the developer to name a unit smaller than what they asked for.

9 of 24 features in each repository owned no behavioural code at all—communities formed entirely from positional gap-bytes that the clustering includes for structural cohesion—and three of those husks carried the names the study’s own removal requests use, so a participant following the map would select a feature and find nothing inside it. The fix (absorb husks into the feature that owns their parent entities) reduced the tree from 24 leaves to 14, but we found it only because a study task routed through it. A purity gate that rejects a domain label when the majority of the group’s symbols are scaffold or test code would fix the 15% label inaccuracy, but we did not build it, because tuning on evaluation data produces a number, not a finding. Figure 7 shows the ratio across all four transcript repositories. No repository reaches 1:1, and the whiskers show that the worst case (13 features for one request) sits on the longest-session history.

8.3 The codebook that was wrong

We coded 60 sampled errors from two repositories (CodeNav and eico) against a pre-registered four-code taxonomy: split (one intent across two features), merge (two intents in one feature), mis-assign (correct groups, wrong membership), and rename (correct membership, wrong label). The pre-registered codes explained 6 of 30 errors in CodeNav and 15 of 30 in eico. On both repositories, the dominant mechanism was a disagreement about what counts as one request, which the clustering handled correctly.

Two mechanisms accounted for the residual. First, **consecutive turns of one feature**: a developer who types “resume” and then “ok sure” between substantive requests creates turn boundaries that the ground truth treats as request boundaries, but sgt groups the

symbols from both turns into one feature, which is the semantically correct answer. On CodeNav this mechanism produced 15 of 30 coded items. Second, **continuation tokens as intent**: on eico, 88% of turns are contentless directives (“resume”, “move on”, “(a)”), and the ground truth attributes each turn’s output to that turn’s text, producing a positive base rate of 47% against a signal that carries no semantic content. sgt correctly refuses to separate work that differs only in which continuation token preceded it.

This finding generalises. Anyone measuring intent recovery against real agentic transcripts will face the same problem, because a per-turn ground truth does not yield a per-turn metric on histories where most turns carry no semantic content. The fraction of contentless turns varies from 0% to 88% across the four repositories we hold, and it is not predictable from session length or repository size. A metric computed on a history with 88% continuation tokens measures the ground truth’s granularity rather than the system’s accuracy.

The one mechanism that is an error in the clustering—intent sharded by directory (F32)—appeared in both repositories and has a consistent shape: two leaves carry the same label and the same intent, separated only because their symbols live in different directories, with no interior node uniting them. This is an addressable defect: a merge pass that detects identically-labelled sibling leaves would eliminate it without changing any parameter.

8.4 Comparison with alternatives

The evaluation data lets us compare against what existing tools offer as well as against our own aspirations. Table 3 summarises the tradeoffs.

Darcs computes dependency from line adjacency [74], so pulling a patch that edits `api.py::handle` without the patch that defined `Retry` in another file produces a reference to a name that does not exist, with no refusal and no warning. sgt’s closure rule prevents this by refusing to write rather than writing a file that does not parse, and the fork rule’s amplification is the cost. The evaluation shows that this conservatism is too expensive on repositories sgt did not record live (27 of 28 refused entirely), but the pilot shows it catching the case it was designed for (two edits to one function from separate requests, neither aware of the other).

sem offers the entity-level answers without the action layer [2], which makes it safe to adopt and safe to abandon. A derived cache that any teammate’s commit can silently invalidate cannot be the record an operation acts on, so sem’s placement is a deliberate choice to trade power for safety. The evaluation suggests this may be the right stopping point for a tool that cannot yet guarantee its write operations: a tool that reports without acting inherits property 1 automatically (there is nothing to corrupt) and sidesteps property 3 entirely (there is no write guard to trigger). The pilot confirms the consequence: the participant independently described sem’s value proposition by rephrasing the revert as “a fast, accurate first pass that leaves me a punch list.” The apparatus pilot showed that the harder problem was reaching the answers at all, since the primary inspect verb rejected 6 of 10 queries (Section 7.4). sem does not face this problem because its commands accept whatever git already accepts (commit hashes, branch names), so a developer’s

Property	Darcs/Pijul	GitButler	sem	sgt
Unit of record	Line-level patch	File-level change	Function (derived cache)	Function (committed record)
Cross-file dep.	Not tracked	Not tracked	Reported, not enforced	Tracked and enforced
Selective revert	Pull/unpull a patch	Undo a file assignment	N/A (read-only)	Revert a named feature
Adoption cost	Replace git	Add to git workflow	None (cache alongside)	Add to git workflow
Failure mode	Silent (line-level)	Silent (file-level)	N/A	Fork rule stops

Table 3: Design tradeoffs across four systems that address change grouping. Darcs and Pijul compute dependency from lines; GitButler assigns files to virtual branches; sem derives a function-level cache but takes no action on it; sgt commits its record and acts on it.

existing vocabulary works without learning a new selector namespace. sgt cannot offer the selective revert, the restore, or the switch from sem’s position, because those operations require writing the subset back to disk—the record must be committed alongside the code so the rebuild is deterministic. Those operations are what motivated sgt’s decision to commit its record, and the evaluation shows they are also where every failure lives. The honest reading of this comparison is that sgt took on an obligation sem declined, and the obligation has not yet paid for itself in the write layer.

GitButler assigns file-level changes to virtual branches [29], which handles the common case where one feature per file suffices. The scenario at the centre of this paper—three requests all editing `api.py:handle`—is the case where file-level assignment cannot express the grouping. sgt’s function-level record handles this case and pays for it with the fork rule, the layout seam, and the identity threshold. The comparison teaches a design lesson: every step downward in granularity (file to function, commit to edit) introduces a new class of consistency problem (layout records, identity tracking, fork amplification) that the coarser grain never had to solve, and the evaluation evidence for whether that cost is justified remains thin—we measured hub absorption (19–41 symbols per shared feature on the study repositories) but not the frequency with which real agent sessions produce the multi-request single-function case that motivates the finer grain. A tool builder choosing a granularity should treat our failure catalogue as a lower bound on the cost of going finer, and GitButler’s adoption as evidence that the coarser grain satisfies most workflows today.

8.5 Design implication: build the read layer before the write layer

The evaluation, the pilot, and the comparison converge on one recommendation for a designer facing the same problem: build the read layer first, ship it alone, and defer the write operations until the developer trusts the reports. Parasuraman’s model explains why. Automating stages one and two (parsing, clustering, dependency display) while leaving stage three to the developer (which feature to name, which revert to confirm) is both safer and independently useful [68]. The pilot confirmed this empirically: the participant adopted the read layer on first contact and declined the write layer after one session. The corpus confirmed it structurally: on 27 of 28 outside repositories, the write layer had nothing to offer because its preconditions were unmet, while the read layer (attribution, dependency display, feature listing) ran on all 35.

The practical consequence is that a tool in this design space should ship its read layer without waiting for write-path stability. The argument has three parts. Thimbleby argued that good design delays commitment until the user holds the information the commitment requires [84], and a developer who has not yet seen the tool’s attributions does not hold the information needed to decide whether to trust its reverts. Schemmer et al. decomposed reliance into two independent measures: the rate at which a user follows the system when it is right (RAIR), and the rate at which they override it when it is wrong (RSR) [77]. A read layer builds RSR directly, because a developer who has seen the attribution can judge whether a proposed revert targets the right set, and override it when it does not. A write layer shipped without that foundation offers no basis for override, and a single visible failure then collapses both metrics—the developer stops following even correct recommendations because they have no independent basis for distinguishing them. But there is a stronger reason. Bainbridge observed that automation degrades the skill it replaces, and the degraded skill is exactly what the operator needs when the automation fails [4]. A developer using an agent to write code no longer practises the decomposition that a revert requires; the read layer (attribution, dependency display, consequence preview) is what restores that decomposition, and without it the developer who needs to intervene is in the position Bainbridge described: lacking the comprehension the automation took from them. Cao et al. found that explanations improve appropriate reliance on automation only when they align with the user’s prior intuition [12]—a preview that names consequences is useful precisely because the read layer has already built the intuition it appeals to (the developer knows what the feature contains because they read the attribution). Without that prior comprehension, the same preview is an assertion the developer cannot evaluate. A read-only release whose attribution is correct builds both the trust and the comprehension that a later write release depends on. Lee and See’s review of trust in automation found that a single visible error erodes trust faster than a string of correct outcomes restores it [55]. Trust is harder to earn after a write failure than before one. This recommendation is in tension with the paper’s own motivation: Section 3 argues for operations (revert, restore, switch) that require writing, and a read-only tool cannot offer them. The resolution is temporal. Ship the read layer now, and ship the write layer when its failure rate is low enough that the trust the read layer earned survives a session. The pilot is evidence that we shipped the write layer too early.

8.6 What would change our conclusion

The result that would overturn this paper’s central claim is an error-rate finding. If a controlled study shows that developers using git alone detect unintended removals at the same rate as developers using sgt’s preview, then the comprehension benefit Section 3 argues for does not exist in practice, and the fork rule, the layout seam, and the naming instability provide no benefit. The pilot cannot settle this because one participant per condition (with the researcher present) measures tool fitness rather than developer behaviour, and the sgt condition’s mutation failures make the comparison one-sided, and the trailer confound (Section 7.4) degrades the git fallback in the treatment arm, so any comprehension advantage we observed could partly reflect a weakened baseline rather than a strengthened treatment. A between-subjects study on tasks where both conditions finish (which the pilot showed they can) would answer it.

Two secondary results would reshape specific claims. First, if the fork rule’s amplification factor holds on repositories recorded live (rather than mined retroactively), then the 0.44 rebuild rate is a property of the design under its intended workflow, and the distinction Section 9.3 draws between the two regimes would collapse. We have partial evidence against this: on the two study repositories, recorded entirely through sgt save, reconstruction is exact (0 drifted paths), which suggests that the fork rule’s cost scales with history length rather than with recording method. Second, if the codebook’s dominant residual—ground-truth grain disagreement—replicates on repositories with more intentful request histories, then the pairwise F1 metric is fundamentally unsuited to this domain and the over-decomposition ratio (2–6× at the median, up to 13) is the only stable measurement available. We report both findings so that future work on intent recovery can calibrate its metrics before discovering the same instability we found.

9 Where the Design Breaks

Every number in this section comes from one run of the command a user would run, on the repository that contains sgt, at a single commit. We took them together rather than as we needed them, because the repository is still being worked on and a set of counts gathered over a week is a set of counts of nothing. That repository holds fifty-seven days and three hundred and forty-six commits of daily use by the person who wrote the system, which is long enough to have found the limits below and not long enough to have found the ones that need a year, and we say which is which where we can tell.

9.1 Most of a repository is not functions

`sgt log --summary` prints 396 file(s), 6821 symbol(s), 154 feature(s), 25% entity coverage, and we printed that last figure for months before checking it against the records it claims to summarise. It divides the files that still hold a live function-level record at the current commit by every path any record mentions, including files deleted long ago, so its two halves are drawn from different populations and the ratio understates the coverage by a factor of nearly three. The count below is the one to read.

Counting the records themselves says something different. This store holds records for 534 files, more than the 396 on disk today

because a record outlives the file it was read from, and of those 534, 381 carry function-level records and 153 are recorded whole. The whole-file set is 82 Markdown files, 19 JSON, 15 $\LaTeX$, 13 HTML, and a scattering of CSS, SVG, and configuration, and it contains no Python and no TypeScript, so every file in a language sgt parses has a function-level record. The design’s coverage of code is complete, and the files left over, nearly three in ten of everything the store has a record for, hold no function for anything to be filed under.

So the limit is the repository’s composition, and calling it a parser problem let us describe a fixed property of software as though it were a defect we could fix. Go and Rust have functions, a tree-sitter grammar plus a decision about which node kinds count as one is a day of work per language, and we would take both tomorrow. It would not move that share at all, because there is no Go and no Rust in it. What is in it is prose, configuration, and generated files, in a proportion no repository of this kind escapes, and a change to four lines of a deployment manifest has no function for the developer’s request to be filed under no matter how many grammars we ship. Every operation in Section 4 still runs on those files, at git’s grain, and none of them gets sharper than `git revert` would be. A reader deciding whether to adopt sgt should ask how much of the work they need to take apart happens inside functions, which is a different question from how much of their repository is code.

9.2 Where a function loses its history

Section 4.1 claims a record survives a rename, and that claim rests on a matcher with a threshold in it. sgt links a function across a commit first by its name, then by identical bytes, then by identical structure with formatting and comments ignored, and last by token overlap above 0.80, with a guard that rejects any pair whose token counts differ by more than a factor of two. The 0.80 comes from sem, which uses the same figure for the same job [2]. It scores the overlap between the two bodies’ sets of distinct tokens, which puts the breaking point lower than the number sounds: replacing more than about one token in nine drops a pair below it, and the function is then recorded as one removed and a different one added.

The developer then sees a function whose recorded history begins at that commit, with its earlier edits still filed under a group the function is no longer part of, and an agent asked to clean up a module produces exactly this, renaming six functions and rewriting their bodies in one sitting. The detectors that get a function’s origin right compare full syntax trees across the two versions [25, 32, 52, 88], where sgt uses the token count because it runs on every save and has to finish while the developer is waiting. Lowering the threshold would link functions that merely resemble each other, and a wrong link is the worse failure, because it files one function’s edits under another function’s name and gives the developer no reason to doubt the answer. sgt advanced identity join states the link by hand, and using it requires the developer to notice that a function’s history restarted, which is the kind of thing they installed sgt in order not to have to notice.

9.3 Fifty-six files we cannot rebuild

The same command reports 56 files whose bytes on disk no valid set of records can produce. The denominator is the 381 files whose records name functions and not the 534 the store holds, because

a file recorded whole holds its own bytes and cannot fail to come back, so one function-level file in seven does not rebuild. `sgt` keeps every one of them, prints the count, names the first five, and points at `sgt advanced fsck --tree`, so nothing was deleted and nothing was quietly wrong (Section 4.2). What the developer loses is the record for those files: a revert of a group that touched one of them leaves the file as it is, and `sgt show` on a function inside it answers from an incomplete chain.

We have not tracked down what put all 56 in that state, and since this store holds records written by five different versions of our miner in fifty-seven days, some of them are the residue of our own format changes. A developer adopting `sgt` on a repository with two years of history should expect a number of this kind on the first run, and should read it as work the design has not finished.

9.4 What a colleague sees

Because the records are files in the repository, everyone who pulls the repository gets them whether they asked for them or not, and this section works through what that means for three people who are not the developer running `sgt`: a colleague who has never installed it, a colleague who installed a different version of it, and the reviewer of a pull request.

The one we worried about most turned out to be the easy case. A colleague who has never heard of `sgt` commits ordinary code, and the next `sgt sync` mines their commits into function-level records on contact, the same way `sgt` mined the history that existed before it was installed. Their edits arrive at the right grain, they group with the work they belong to, and nothing has to be repaired. What does not arrive is the request, because there was no request to capture: their records carry no sentence, so the name for their group is drawn from their commit subjects, and where one subject accounts for most of a group `sgt` uses that subject as the name. A colleague who writes `fix tests` and `wip` gets a group named from the code instead, which is the same failure Section 4.4 describes for a save with no message. So the grain `sgt` recovers from anybody and the request only from somebody who was running it, and a team that half-adopts `sgt` gets a history that is half named.

The harder case is a colleague who does install it and runs a different version. `sgt` recovers its records from code instead of being handed them, so the program that did the reading is part of what a record is, and an edit's identifier is computed from its content together with the version of the miner that produced it. Two versions therefore mint different identifiers for the same edit, and a `sync` across that gap would quietly alias records that mean different things, which `sgt` avoids by refusing the whole union and printing what to do. Two people using `git` have to agree about bytes and nothing else, and two people using `sgt` have to agree about a program. That refusal is correct and it is also the cost we are least able to speak to, because Section 9.3 reports five miner versions inside fifty-seven days and every one of those transitions would have stopped a collaborator. It stopped nobody, since there was nobody to stop, and a design decision that a solo repository charges nothing for is a decision that repository cannot evaluate.

The third case is the reviewer, and here we have no answer at all. A pull request that carries `sgt` records shows them as what they are on disk, a few hundred JSON files whose names are hashes, and no

reviewer reads that; they review the code, as they should, and how anybody should read a large model-written change spanning many files is an open question being studied on its own [40]. So the record that makes revert work is not part of what anybody approved, and a resolution one developer reasoned through and a label another corrected both enter the shared branch unexamined. The rule in Section 4.3 asks a person to settle a fork, and on a team that person is still one person, while the review that would have caught them being wrong reads a diff their decision does not appear in.

9.5 The fork rule has never met two people

Section 4.3 says a resolved fork has to pass the project's test command before `sgt` will include it, and the summary above also prints oracle: unconfigured. In the repository we use every day, resolving a fork means the developer picks a version and `sgt` takes their word for it, because nobody ever pointed `sgt` at our test runner. A safety property a developer has to configure is a safety property most repositories will not have, so the strongest claim in Section 4.3 holds only where somebody did that setup, and the person who wrote the rule did not do it in the repository where the rule runs every day. The fix needs no research, because `sgt` could find the test command the project already uses and offer it during `sgt init`, and making the developer ask for it was the wrong default.

The condition the rule has been tested under is narrower still. This repository's history carries three git identities and one human being behind all of them, so every merge `sgt` has ever seen had the same person on both sides of it, which is the condition in which the rule is most obviously right, because the person a fork stops is the person who wrote both versions and can answer it in seconds. It is also the condition in which the rule is cheapest. On a repository where twenty people edit the same functions every day, the same rule produces a queue of decisions whose two sides were written by people who have not spoken, and Nelson et al.'s account of how teams work through merge conflicts describes the barriers a queue of that kind meets [65]. We expect that team would switch the rule off and take the textual merge, and our evidence cannot tell them not to.

The setting we would build applies `git`'s rule at the grain of a function, merging when the two edits changed no overlapping lines and asking when they did, which would also let `sgt` report a divergence while both sides are still being written rather than when they meet [36]. We have not built it, because we cannot tell a reader which of the two should be the default, and the repository we would have tested it on has one person in it. This is the part of the design a study of larger teams would most usefully overturn.

9.6 The record the agent already had

The grouping `sgt` recovers by clustering existed before any of the code did. The agent held it while it worked, and every agent we have used discards it when the session ends. A record written by the agent, one entry per request naming the functions it changed, would make the inference in Section 4.4 unnecessary, and it would fix the case in Section 5 where a rename nobody asked for got a name derived from the code.

Half of that record we do take, and the half we take is the half a vendor cannot spoil. Where the developer's agent is Claude Code,

sgt installs a hook on prompt submission and stores every sentence the developer typed, with its time, so the words that named the work are on disk before any code exists and sgt now answers with what was asked for rather than with a count of edits. That is twenty lines and one vendor's hook interface, and the same twenty lines have to be written again for the next agent, which is the price we decided we could pay, because what they capture is the developer's own English and a change of vendor cannot make yesterday's sentence wrong.

The half we do not take is the agent's account of which functions each request touched, and the reason is that a record of that kind is a claim about the files which we would have to either trust or check. Trusting it puts the correctness of a revert inside a transcript nobody validated against the code. Checking it means reading the files and computing the answer, at which point we did not need the account. What the agent's account would improve is the label, which is the part of sgt that is allowed to be wrong, and improving a label is worth a hint and not worth a dependency. So the split we have arrived at is that membership is read from code and names are read from words, and the words come from the hook where there is a hook and from the save message where there is not. What we do not know is whether an agent's account of its own edits is accurate enough to move membership too, and that is the measurement we would ask a reader of this paper to make before any other.

Open science

sgt is open source, and the repository it versions is the repository that implements it, so the artefact and the evidence in this section are the same object: a reader can clone it, check out the commit the artefact names, run `sgt log --summary`, and read every number in this section off the output. Run at a later commit the counts will have moved, since the repository is the working one. The between-subjects study in Section 6.3 is pre-registered: its design, measures, and the thresholds in Table 1 were fixed before recruiting, so that the first row remains the first row after we have seen the data. Two moderated pilots (Section 7) validated the tasks, the rubric, and the scoring script; they found twelve tool defects and four design defects, all fixed before the full study begins. The two repositories the study uses, the transcripts that produced them, and the correctness script will be released with the results. The study's only recorded data will be screen activity, task outcomes, and interview audio from consenting professional developers, with no personal data beyond what consent and compensation require.

10 Conclusion

We set out to build a version control system that records the edits a developer asks for instead of the lines an agent writes, and to test that system honestly. The evaluation found that the read operations work and the write operations fail. Attribution, preview, and dependency display survived first contact with a user who had never seen the tool, and a participant in the baseline condition independently described the capability the tool provides. The write operations did not survive. On 27 of 28 repositories we did not record ourselves, the primary verbs refused to run because they could not guarantee the rebuilt file would match what the developer left on disk. The refusal is correct, but it converts a tool that promises selective

revert into a tool that declines to record the next edit. The cause is the fork rule. The fork rule withdraws records to protect 162 actually-forked functions. That withdrawal cascades eighteen-fold to cover 2,864 function keys, the rebuild becomes incomplete, and every write guard fires on the gap. The fix is straightforward in principle (loosen the fork rule) but involves a real tradeoff: loosening it gives up the safety guarantee of Section 4.3 in exchange for the completeness of Section 6.1.

Section 8 traces each design commitment back to its evaluation outcome. The pattern across all four commitments is the same: the part that shows information to the developer held up under first contact, and the part that modifies the developer's files either failed or refused to act.

Three findings generalise past this system. First, automating the earlier stages of Parasuraman's model (parsing, clustering, dependency tracking) delivers value independently of whether the later stages work, and a developer who trusts the reports can use them with git directly. Second, the failure mode that cost us the most was a command that reported success while doing nothing. A silent lie is worse than a crash because a tool that lies quietly erodes the basis for trusting anything else it says, and that erosion did not recover within a session (Section 7.5). Any tool that mediates between a developer and code they did not write faces this same constraint: the developer's inability to verify independently is the reason the tool exists, and a tool whose reports cannot be trusted leaves the developer worse off than having no tool, because they have spent their verification budget on a false assurance. Third, evaluating intent recovery against real agentic transcripts is unstable in a way the metric hides. The share of contentless turns varies from 0% to 88% across our four repositories, and a pairwise F1 computed on a history with 88% continuation tokens measures the ground truth's grain rather than the system's accuracy (Section 8.3).

The second finding generalises further. Durability and legibility require different kinds of guarantee. The append-only store ensures that every prior state is reachable, and it does so structurally. But the developer still has to know which command to type and whether its output told the truth, and the architecture provides no structural guarantee of that. Every one of our seven self-reporting failures demonstrated this gap: the data was present, and the tool's account of how to reach it was wrong.

References

- [1] Sven Apel, Jörg Liebig, Benjamin Brandl, Christian Lengauer, and Christian Kästner. 2011. Semistructured merge: rethinking merge in revision control systems. In *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 190–200.
- [2] Ataraxy Labs. 2026. sem: Semantic version control built on Git. <https://github.com/ataraxy-labs/sem>. Accessed 2026.
- [3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617
- [4] Lisanne Bainbridge. 1983. Ironies of automation. *Automatica* 19, 6 (1983), 775–779. doi:10.1016/0005-1098(83)90046-8
- [5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. doi:10.1145/3586030
- [6] Mike Barnett, Christian Bird, João Brunet, and Shuvendu K. Lahiri. 2015. Helping developers help themselves: Automatic decomposition of code review change-sets. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 134–144. doi:10.1109/ICSE.2015.35

- [7] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakçı, and Rei Mariman. 2025. Generative AI can harm learning. *Proceedings of the National Academy of Sciences* 122, 26 (2025), e242633122. doi:10.1073/pnas.2422633122
- [8] Ted J. Biggerstaff, Bharat G. Mithbander, and Dallas Webster. 1993. The concept assignment problem in program understanding. In *Proceedings of the 15th International Conference on Software Engineering (ICSE)*. 482–498. doi:10.1109/ICSE.1993.346017
- [9] Alan F. Blackwell and Thomas R. G. Green. 2003. Notational systems: The Cognitive Dimensions of Notations framework. In *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, John M. Carroll (Ed.). Morgan Kaufmann, 103–134.
- [10] Max Brunsfeld and the tree-sitter contributors. 2026. tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/>. Accessed 2026.
- [11] Raymond P. L. Buse and Westley R. Weimer. 2010. Automatically documenting program changes. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 33–42. doi:10.1145/1858996.1859005
- [12] Ruijia Cao, Zhuoren Lu, and Chun-Wei Huang. 2024. Toward appropriate reliance: complementary explanations reduce both over- and under-reliance on AI. *Proceedings of the ACM on Human-Computer Interaction* 8, CSCW2 (2024). doi:10.1145/3687054
- [13] John M. Carroll and Mary Beth Rosson. 1987. Paradox of the active user. *Interfacing Thought: Cognitive Aspects of Human-Computer Interaction* (1987), 80–111.
- [14] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. “It would work for me too”: How online communities shape software developers’ trust in AI-powered code generation tools. *ACM Transactions on Interactive Intelligent Systems* (2024). doi:10.1145/3651990
- [15] Mark C. Chu-Carroll and Sara Sprenkle. 2000. Coven: brewing better collaboration through software configuration management. In *Proceedings of the 8th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 88–97.
- [16] Mark C. Chu-Carroll, James Wright, and David Shields. 2002. Supporting aggregation in fine grained software configuration management. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 99–108.
- [17] Luke Church, Emma Söderberg, and Bhargava Elango. 2014. A case of computational thinking: The subtle effect of hidden dependencies on the user experience of version control. In *Proceedings of the 25th Annual Workshop of the Psychology of Programming Interest Group (PPiG)*.
- [18] Mihai Codoban, Sruti Srinivasa Ragavan, Danny Dig, and Brian Bailey. 2015. Software history under the lens: A study on why and how developers examine it. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–10. doi:10.1109/ICSM.2015.7332446
- [19] Zheyuan (Kevin) Cui, Mert Demirer, Sonia Jaffe, Leon Musolf, Sida Peng, and Tobias Salz. 2025. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *SSRN Electronic Journal* (2025). doi:10.2139/ssrn.4945566
- [20] Martin Dias, Alberto Bacchelli, Georgios Gousios, Damien Cassou, and Stéphane Ducasse. 2015. Untangling fine-grained code changes. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 341–350. doi:10.1109/SANER.2015.7081844
- [21] Danny Dig, Kashif Manzoor, Ralph Johnson, and Tien N. Nguyen. 2007. Refactoring-aware configuration management for object-oriented programs. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. 427–436.
- [22] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: A taxonomy and survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567
- [23] Ian Drosos and Advait Sarkar. 2025. Vibe coding: Programming through conversation with artificial intelligence. *arXiv preprint arXiv:2506.23253* (2025).
- [24] Ruoxi Duan, Jina Suh, Alex Gu, Zijian Liu, Hussein Mozannar, and Gagan Bansal. 2026. I don’t trust it because my friend said so: trust contagion in human-AI teams. In *Proceedings of the 2026 ACM Conference on Computer-Supported Cooperative Work and Social Computing*. doi:10.1145/3706598.3713903
- [25] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*. 313–324. doi:10.1145/2642937.2642982
- [26] Kasra Ferdowsi, Ruanqianqian Huang, Madeline B. James, Nadia Polikarpova, and Sorin Lerner. 2024. Validating AI-generated code with live programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642495
- [27] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald C. Gall. 2007. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. doi:10.1109/TSE.2007.70731
- [28] George W. Furnas, Thomas K. Landauer, Louis M. Gomez, and Susan T. Dumais. 1987. The vocabulary problem in human-system communication. *Commun. ACM* 30, 11 (1987), 964–971. doi:10.1145/32206.32212
- [29] GitButler. 2026. GitButler documentation: Parallel branches. <https://docs.gitbutler.com/features/virtual-branches/virtual-branches>. Accessed 2026.
- [30] GitClear. 2025. AI copilot code quality: 2025 look back at 12 months of data. https://www.gitclear.com/ai_assistant_code_quality_2025_research. 211 million changed lines of code authored January 2020 to December 2024.
- [31] GitHub. 2026. Stacked pull requests are now in public preview. <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>. Accessed 2026.
- [32] Michael W. Godfrey and Lijie Zou. 2005. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering* 31, 2 (2005), 166–181. doi:10.1109/TSE.2005.28
- [33] Graphite. 2026. Stacked diffs and how to use them. <https://graphite.com/guides/stacked-diffs>. Accessed 2026.
- [34] Thomas R. G. Green and Marian Petre. 1996. Usability analysis of visual programming environments: A ‘cognitive dimensions’ framework. *Journal of Visual Languages and Computing* 7, 2 (1996), 131–174. doi:10.1006/jvlc.1996.0009
- [35] Tovi Grossman, Justin Matejka, and George Fitzmaurice. 2010. Chronicle: Capture, exploration, and playback of document workflow histories. In *Proceedings of the 23rd Annual ACM Symposium on User Interface Software and Technology (UIST)*. 143–152. doi:10.1145/1866029.1866054
- [36] Mário Luis Guimarães and António Rito Silva. 2012. Improving early detection of software merge conflicts. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 342–352. doi:10.1109/ICSE.2012.6227180
- [37] Björn Hartmann, Loren Yu, Abel Allison, Yeonsoo Yang, and Scott R. Klemmer. 2008. Design as exploration: Creating interface alternatives through parallel authoring and runtime tuning. In *Proceedings of the 21st Annual ACM Symposium on User Interface Software and Technology (UIST)*. 91–100. doi:10.1145/1449715.1449732
- [38] Hideaki Hata, Osamu Mizuno, and Tohru Kikuno. 2011. Historage: fine-grained version control system for Java. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWSE-EVOL)*. 96–100.
- [39] Andrew Head, Fred Hohman, Titus Barik, Steven M. Drucker, and Robert DeLine. 2019. Managing messes in computational notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3290605.3300776
- [40] Lars Heander, Agnia Sergeyuk, Ilya Zakharov, et al. 2026. Trust-calibrated code review: A participatory design study of review workflows for LLM-generated multi-file changes. *arXiv preprint* (2026).
- [41] Kim Herzog, Sascha Just, and Andreas Zeller. 2016. The impact of tangled code changes on defect prediction models. *Empirical Software Engineering* 21, 2 (2016), 303–336. doi:10.1007/s10664-015-9376-6
- [42] Kim Herzog and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. 121–130. doi:10.1109/MSR.2013.6624018
- [43] Yoshiaki Higo, Shinpei Hayashi, and Shinji Kusumoto. 2020. On tracking Java methods with Git mechanisms. *Journal of Systems and Software* 165 (2020), 110571.
- [44] Susan Horwitz, Jan Prins, and Thomas Reps. 1989. Integrating noninterfering versions of programs. *ACM Transactions on Programming Languages and Systems* 11, 3 (1989), 345–387.
- [45] Daniel Jackson. 2021. *The Essence of Software: Why Concepts Matter for Great Design*. Princeton University Press.
- [46] Judah Jacobson. 2009. A formalization of Darcs patch theory using inverse semigroups. *Computational and Applied Mathematics Reports, University of California, Los Angeles* (2009). CAM report 09-83.
- [47] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [48] Majeed Kazemitabaar, Olivia Huang, Sangho Suh, Austin Z. Henley, and Tovi Grossman. 2025. Exploring the design space of cognitive engagement techniques with AI-generated code for enhanced learning. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*. doi:10.1145/3708359.3712104
- [49] Mik Kersten and Gail C. Murphy. 2006. Using task context to improve programmer productivity. In *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–11. doi:10.1145/1181775.1181777
- [50] Mary Beth Kery, Amber Horvath, and Brad A. Myers. 2017. Variolite: Supporting exploratory programming by data scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 1265–1276. doi:10.1145/3025453.3025626
- [51] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–11. doi:10.1145/3173574.3173748

- [52] Sunghun Kim, Kai Pan, and E. James Whitehead. 2005. When functions change their names: Automatic detection of origin relationships. In *Proceedings of the 12th Working Conference on Reverse Engineering (WCRE)*. 143–152. doi:10.1109/WCRE.2005.33
- [53] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987. doi:10.1109/TSE.2006.116
- [54] Thomas D. LaToza and Brad A. Myers. 2010. Hard-to-answer questions about code. In *Evaluation and Usability of Programming Languages and Tools (PLATEAU)*. 1–6. doi:10.1145/1937117.1937125
- [55] John D. Lee and Katrina A. See. 2004. Trust in automation: Designing for appropriate reliance. *Human Factors* 46, 1 (2004), 50–80. doi:10.1518/hfes.46.1.50.30392
- [56] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13. doi:10.1145/3597503.3608128
- [57] Ernst Lippe and Norbert van Oosterom. 1992. Operation-based merging. In *Proceedings of the Fifth ACM SIGSOFT Symposium on Software Development Environments (SDE)*. 78–87.
- [58] Poornima Madhavan and Douglas A. Wiegmann. 2007. Similarities and differences between human–human and human–automation trust: An integrative review. *Theoretical Issues in Ergonomics Science* 8, 4 (2007), 277–301. doi:10.1080/14639220500337708
- [59] Pierre-Étienne Meunier and Florent Becker. 2024. The Pijul manual. <https://pijul.org/manual/>. Accessed 2026.
- [60] Hiroaki Mikami, Daisuke Sakamoto, and Takeo Igarashi. 2017. Micro-versioning tool to support experimentation in exploratory programming. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 6208–6219. doi:10.1145/3025453.3025597
- [61] Samuel Mimram and Cinzia Di Giusto. 2013. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science* 298 (2013), 283–307. doi:10.1016/j.entcs.2013.09.018
- [62] Scott Mishler and Jamy Chen. 2023. The effect of automation errors on trust: A meta-analysis. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, Vol. 67. 279–284. doi:10.1177/21695067231192235
- [63] Hussein Mozannar, Valerie Chen, Mohammed Alsobay, Subhro Das, Sebastian Zhao, Dennis Wei, Manish Nagireddy, Prasanna Sattigeri, Ameet Talwalkar, and David Sontag. 2024. The RealHumanEval: Evaluating large language models’ abilities to support programmers. *arXiv preprint arXiv:2404.02806* (2024).
- [64] Peter Naur. 1985. Programming as Theory Building. *Microprocessing and Microprogramming* 15, 5 (1985), 253–261. Reprinted in *Computing: A Human Activity*. ACM Press, 1992.
- [65] Nicholas Nelson, Caius Brindescu, Shane McKee, Anita Sarma, and Danny Dig. 2019. The life-cycle of merge conflicts: Processes, barriers, and strategies. *Empirical Software Engineering* 24, 5 (2019), 2863–2906. doi:10.1007/s10664-018-9674-x
- [66] Tuan Anh Nguyen and Elena L. Glassman. 2024. How beginning programmers and code LLMs (mis)read each other. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642706
- [67] Donald A. Norman. 1988. *The Design of Everyday Things*. Basic Books. Originally published as *The Psychology of Everyday Things*, 1988; revised edition 2013.
- [68] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30, 3 (2000), 286–297. doi:10.1109/3468.844354
- [69] Santiago Perez De Rosso and Daniel Jackson. 2013. What’s wrong with git? A conceptual design analysis. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*. ACM, 37–52. doi:10.1145/2509578.2509584
- [70] Santiago Perez De Rosso and Daniel Jackson. 2016. Purposes, concepts, misfits, and a redesign of git. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 292–310. doi:10.1145/2983990.2984018
- [71] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [72] Profir-Petru Pârțachi, Santanu Kumar Dash, Christoph Treude, and Earl T. Barr. 2020. Flexeme: Untangling commits using lexical flows. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 63–74. doi:10.1145/3368089.3409693
- [73] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 202–212. doi:10.1145/2491411.2491444
- [74] David Roundy. 2005. Darcs: Distributed version management in Haskell. In *Proceedings of the 2005 ACM SIGPLAN Workshop on Haskell*. 1–4. doi:10.1145/1088348.1088349
- [75] S. Sabouri, P. Eibl, X. Zhou, M. Ziyadi, N. Medvidovic, L. Lindemann, and S. Chattopadhyay. 2025. Trust dynamics in AI-assisted development: definitions, factors, and implications. In *Proceedings of the 47th International Conference on Software Engineering*. doi:10.1109/ICSE55347.2025.11029928
- [76] Advait Sarkar, Andrew D. Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group (PPIG)*.
- [77] Max Schemmer, Niklas Kühl, Carina Benz, and Gerhard Satzger. 2022. Appropriate reliance on AI advice: conceptualization and the effect of explanations. In *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society*. doi:10.1145/3514094.3534089
- [78] Bo Shen, Wei Zhang, Christian Kästner, Haiyan Zhao, Zhao Wei, Guangtai Liang, and Zhi Jin. 2021. SmartCommit: A graph-based interactive assistant for activity-oriented commits. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 379–390. doi:10.1145/3468264.3468551
- [79] Frank M. Shipman and Catherine C. Marshall. 1999. Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representations in Interactive Systems. *Computer Supported Cooperative Work* 8, 4 (1999), 333–352.
- [80] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. Developer behaviors in validating and repairing LLM-generated code using IDE and eye tracking. In *Proceedings of the 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 40–46.
- [81] Yida Tao and Sunghun Kim. 2015. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*. 180–190. doi:10.1109/MSR.2015.24
- [82] Michael Terry, Elizabeth D. Mynatt, Kumiyo Nakakoji, and Yasuhiro Yamamoto. 2004. Variation in element and action: Supporting simultaneous development of alternative solutions. In *Proceedings of the 2004 CHI Conference on Human Factors in Computing Systems*. 711–718. doi:10.1145/985692.985782
- [83] The Mercurial contributors. 2026. Mercurial wiki: Changeset evolution. <https://wiki.mercurial-scm.org/ChangesetEvolution>. Accessed August 2026.
- [84] Harold Thimbleby. 1988. Delaying commitment. *IEEE Software* 5, 3 (1988), 78–86. doi:10.1109/52.2027
- [85] Linus Torvalds, Junio C. Hamano, and the Git contributors. 2026. Git: Distributed version control system. <https://git-scm.com/>. Accessed 2026.
- [86] V. A. Traag, P. Van Dooren, and Y. Nesterov. 2011. Narrow scope for resolution-limit-free community detection. *Physical Review E* 84 (2011), 016114. doi:10.1103/PhysRevE.84.016114
- [87] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233. doi:10.1038/s41598-019-41695-z
- [88] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 483–494. doi:10.1145/3180155.3180206
- [89] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–7. doi:10.1145/3491101.3519665
- [90] Helena Vasconcelos, Gagan Bansal, Adam Fournay, Q. Vera Liao, and Jennifer Wortman Vaughan. 2025. Generation probabilities are not enough: Uncertainty highlighting in AI code completions. *ACM Transactions on Computer-Human Interaction* (2025). doi:10.1145/3702320
- [91] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Comparison with Git. <http://docs.jj-vcs.dev/latest/git-comparison/>. Accessed August 2026.
- [92] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Glossary. <http://docs.jj-vcs.dev/latest/glossary/>. Accessed August 2026.
- [93] Min Wang, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2019. CoRA: Decomposing and describing tangled code changes for reviewer. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1050–1061. doi:10.1109/ASE.2019.00101
- [94] Ruotong Wang, Ruijia Peng, Chin-Wei Cheng, Yun-Chien Chen, Chia-Yi Kuo, and Hao-Chuan Hsieh. 2024. Trust calibration in AI-assisted code generation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642503
- [95] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [96] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE Transactions on Software Engineering* 30, 9 (2004), 574–586. doi:10.1109/TSE.2004.52
- [97] YoungSeok Yoon and Brad A. Myers. 2014. A longitudinal study of programmers' backtracking. In *Proceedings of the 2014 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 101–108. doi:10.1109/VLHCC.2014.6883030
- [98] YoungSeok Yoon and Brad A. Myers. 2015. Supporting selective undo in a code editor. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 223–233. doi:10.1109/ICSE.2015.43
- [99] YoungSeok Yoon, Brad A. Myers, and Sebon Koo. 2013. Visualization of fine-grained code change history. In *Proceedings of the 2013 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 119–126. doi:10.1109/VLHCC.2013.6645254
- [100] Andreas Zeller and Gregor Snelting. 1997. Unified versioning through feature logic. *ACM Transactions on Software Engineering and Methodology* 6, 4 (1997), 398–441.
- [101] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining version histories to guide software changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. 563–572. doi:10.1109/ICSE.2004.1317478